TECHNICAL REFERENCE & MAINTAINER MANUAL

RXT-J+ Fraud Detection System

Complete Project Documentation

Repository root	`/home/vishu/Projects/rxtj_phase_2`
Primary language	Python 3 (FastAPI / PyTorch / scikit-learn), Jupyter notebooks, static HTML/JS
Document generated	2026-09-16
Generation method	Full static analysis of source, notebooks, serialized model artifacts, and configuration files. No repository content was invented; unavailable information is explicitly labeled.

This document was produced by direct inspection of the repository: reading all source files, parsing all eight training notebooks, deserializing all eleven model artifacts (4 PyTorch state dicts, 8 scikit-learn/joblib objects) with their exact tensor shapes and fitted parameters, reading the Parquet snapshot schema, and materializing the SQLite schema from the store's DDL. Where the project's own `README.md` disagrees with the executable code or serialized artifacts, both are reported and the discrepancy is flagged explicitly rather than silently resolved.

Table of Contents

1. Project Overview **3**

2. Complete File Inventory **8**

3. Complete Class & Function Catalog **16**

4. Library & Tool Inventory **24**

5. Complete Data Flow & Workflows **27**

5.1 Phase 1 Training Pipeline 27

5.2 Phase 2 Training Pipeline 29

5.3 Inference Pipeline (/score/form-full) 33

6. Model Architectures **36**

7. Configuration & Artifacts **41**

8. API Specification **46**

9. Database Schema **51**

10. Notebook Execution Order **56**

11. Known Issues / TODO / Technical Debt **59**

12. Deployment & Operations **65**

13. Security Analysis of the Implementation **69**

14. Explainability & Decision Logic **72**

15. Artifact Lifecycle **75**

16. Complete Cross-Reference Index **78**

17. Final Validation **82**

1. Project Overview

1.1 Identity

Project name	**RXT-J+** — README title: "RXT-J+ · Real-Time Transaction Risk Scoring"; FastAPI app title (`app.py:314`): "RXT-J+ Fraud Risk Scoring API", version `2.0.0`.
Purpose	Real-time machine-learning risk scoring for (a) individual payment transactions and (b) whether the account behind a transaction has been taken over ("account compromise").
Problem being solved	Credit-card / payment fraud detection and account-takeover (ATO) detection on the IEEE-CIS Fraud Detection dataset (590,540 transactions), extending a published baseline (RXT-J, Almazroi & Ayub, IEEE Access 2023, cited in README.md) with non-linear feature extraction, self-attention, and an added account-level behavioral-fusion stage.
Fraud-detection objective	Two objectives, implemented as two sequential model stages: (1) per-transaction fraud probability ("Phase 1" / "P1"), and (2) per-account compromise probability conditioned on behavioral history ("Phase 2").

> **DOCUMENTATION VS. REPOSITORY**
>
> `README.md` is a template-style document containing maintainer placeholder comments (e.g. "GRAPHICS NOTE FOR MAINTAINERS", shields.io badge instructions) and describes tooling that does not exist in this repository: a `tests/` directory, `.env.example`, `Dockerfile`, `docker-compose.yml`, a `.github/workflows/` CI pipeline, a `docs/assets/` directory, and a script named `fix_p1_scores.py`. None of these paths exist on disk. The README also describes an earlier architecture revision (FusionNet **v2**, 8 contextual features, 8→64→32→16→1) that has been superseded in the executable code by FusionNet **v3** (13 features, 13→128→64→32→1, confirmed both in `app.py:153-180` and in the `fusion_net.pt` state dict). This document describes the system *as implemented*, and calls out every place the README diverges from it.

1.2 High-level architecture

The system has two trained scoring stages that share a common feature-preprocessing front end, a FastAPI serving layer, a SQLite-backed behavioral profile store, an optional Kafka producer/consumer pair, and a set of static HTML demo front ends.

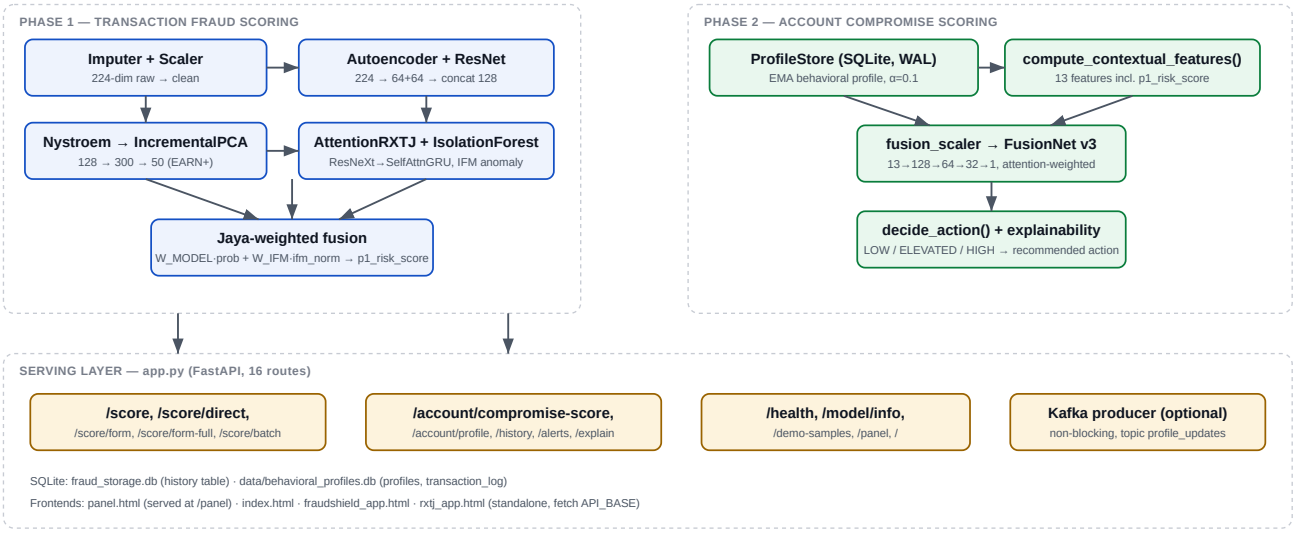


Figure 1.1 — High-level architecture: Phase 1 transaction scoring, Phase 2 account-compromise scoring, and the FastAPI serving layer that fronts both (drawn from app.py load_models() / route table, profile_store.py, and notebooks 03–08).

1.3 Phase 1 architecture

Phase 1 (per-transaction fraud scoring) is a five-stage pipeline, trained across notebooks 01–05 and re-implemented verbatim inside `app.py` for inference (see §5.1 and §6 for full detail):

1. **Imputer + Scaler** — `sklearn.impute.SimpleImputer(strategy='mean')` and `sklearn.preprocessing.StandardScaler`, both fit on 224 numeric/label-encoded columns of the IEEE-CIS `train_transaction.csv` (confirmed via `imputer.pkl`: `n_features_in_=224`).
2. **EARN+ feature extraction** — an `AutoencoderNet` (224→64 latent) and a `FraudResNet` (224→64 features) run in parallel on the same scaled input; their outputs are concatenated to 128 dimensions.
3. **Nystroem + IncrementalPCA** — a Nystroem RBF kernel approximation (128→300) followed by IncrementalPCA (300→50) reduces the concatenated embedding to a 50-dimensional "EARN+" feature vector (`Z_earn`).
4. **AttentionRXTJ + IsolationForest** — a ResNeXt feature extractor feeds a 2-layer self-attention GRU classifier (`AttentionRXTJ`), producing a fraud probability; an `IsolationForest` trained on the same 50-dim space produces a normalized anomaly score.
5. **Jaya-optimized fusion** — the two scores are combined as `risk = (W_MODEL/total)·prob + (W_IFM/total)·ifm_norm`, with weights and decision threshold selected in notebook 05 by a Jaya (gradient-free, single-hyperparameter-free) metaheuristic optimizer minimizing a fiscal-cost objective.

1.4 Phase 2 architecture

Phase 2 (account compromise scoring) sits on top of Phase 1 and adds account-level behavioral context, trained in notebooks 06–08:

1. **Behavioral profiling** (NB06) — builds one row per account (`card1`) in a SQLite `profiles` table, updated incrementally with an exponential moving average (α=0.1) over amount, hour-of-day histogram, merchant/product-code counts, known devices, and velocity counters.
2. **Contextual features** (NB07) — computes 13 per-transaction contextual/deviation features (amount z-score, merchant novelty, geo displacement, hour deviation, device novelty, velocity ratio, autoencoder-based behavioral drift, Phase-1 risk score, amount percentile, time-gap norm, merchant fraud rate, cross-device-account ratio, account-age score) and an account-compromise label (fraud transaction OR within a 24h pre-fraud "probe window").
3. **FusionNet v3** (NB08) — a 13→128→64→32→1 MLP with a learnable per-feature softmax attention gate, trained with Focal Loss (α=0.75, γ=2.0) on a *temporal* (not random) 70/15/15 train/val/test split, followed by a second Jaya threshold search over the validation set.

1.5 How Phase 1 and Phase 2 interact

Phase 2 is strictly downstream of Phase 1: the Phase-1 risk score (`p1_risk_score`) is one of the 13 Phase-2 input features (feature index 7, see `app.py:456-465`), and the same autoencoder used inside Phase 1's EARN+ extraction is reused at inference time to compute the Phase-2 `behavioral_drift` feature via reconstruction error (`app.py:507-514`). Both endpoints that run the full pipeline (`/score/form-full`, `/account/compromise-score`) therefore execute the entire Phase 1 pipeline first and feed its output into Phase 2. Phase 1 can run standalone (`/score`, `/score/direct`, `/score/form`, `/score/batch`); Phase 2 cannot run without a Phase-1 score for the `p1_risk_score` feature.

1.6 Training architecture vs. inference architecture

Training (notebooks 01–08, run once, offline, against the full 590,540-row IEEE-CIS CSV) produces eleven serialized artifacts in `models/` plus JSON/PNG artifacts in `data/` and `results/`. Inference (`app.py`, run continuously as a FastAPI service) loads all eleven artifacts once at process startup (`app.py:185-231`, function `load_models()`) and re-declares the exact same PyTorch module classes used in training so that `state_dict` keys line up. The inference-side model classes are near-duplicates of the notebook-side classes with one architecturally significant difference documented in the source itself: the header comment in `app.py:6-9` records that an earlier version of `FusionNet` had a 32→16→1 tail that did not match the trained weights, and the current 32→1 (no intermediate 16) tail was corrected to match `fusion_net.pt`.

1.7 Profile system

The behavioral profile system is `profile_store.py`'s `ProfileStore` class: a SQLite database in WAL mode with two tables (`profiles`, one row per account; `transaction_log`, one row per scored event). Profiles are updated with an EMA (α=0.1, module constant `_EMA_ALPHA`) after every call to `/score/form-full` or `/account/compromise-score`. See §9 for the full schema and §3 for every method.

1.8 Database

Two separate SQLite databases exist at runtime, both gitignored (`*.db` in `.gitignore:26`) and therefore absent from the repository as delivered:

- `fraud_storage.db` — created by `app.py:_init_history_db()` (app.py:257-267), single table `history`, used only by the legacy `/score/form` and `/history` endpoints.
- `data/behavioral_profiles.db` — created and owned by `ProfileStore`, holds the `profiles` and `transaction_log` tables that back all Phase-2 endpoints.

1.9 Kafka integration

Kafka is an optional, non-blocking event stream. app.py:280-309 constructs a `kafka.KafkaProducer` only if the environment variable `KAFKA_BOOTSTRAP_SERVERS` is set and non-empty; every call to `/score` then publishes a JSON event to the `profile_updates` topic (default name, overridable via `KAFKA_PROFILE_TOPIC`) via `_publish_profile_update()`. profile_consumer.py is a standalone script that consumes the same topic and currently only logs each event to stdout — its `handle_event()` function is explicitly a stub ("Override / extend this with the Phase 2 profile-store update logic", profile_consumer.py:24); it does not itself call `ProfileStore`. Profile updates in the running system are in fact performed synchronously inside the FastAPI request handlers (`profile_store.update_profile(...)` in app.py:889, 1008), not via the Kafka consumer.

1.10 API layer

app.py is a single-file FastAPI application (1,151 lines) exposing 16 routes (see §8 for the complete specification). CORS is fully open (`allow_origins=["*", "null"]`, app.py:319-325) to support the `file://`-opened demo HTML pages. There is no authentication or authorization on any route.

1.11 Explainability system

Explainability is implemented two ways: (1) FusionNet's own learnable `feature_attn` parameter (a softmax-normalized weight per contextual feature, app.py:162,176-180) is returned as the `explainability` dict on every Phase-2 scoring call, with the highest-weighted feature reported as `top_trigger_feature`; (2) `/account/explain` takes a previously-logged event and produces a natural-language explanation by combining the stored context features with a hand-written `interpret()` lookup table (app.py:1094-1111). See §14.

1.12 Key metrics actually present in the repository

Metric	Value	Source file
Phase 1 ROC-AUC	0.962440	`results/deployment_config.json` → `final_auc`
Phase 1 MCC	0.842620	`results/deployment_config.json` → `final_mcc`
Phase 1 false positives / negatives (test, n=82,484)	2,748 / 3,753	`results/deployment_config.json`
Phase 1 throughput / latency	51,351.76 txn/s · 0.019474 ms/txn	`results/deployment_config.json` — **see §11.1: measured on a synthetic/partial pipeline, not the full inference path**
Phase 2 (FusionNet v3) test ROC-AUC	0.740583	`results/fusion_config.json` → `fusion_auc`
Phase 2 test PR-AUC	0.102779	`results/fusion_config.json` → `fusion_pr_auc`

Phase 2 test MCC	0.186221	`results/fusion_config.json` → `fusion_mcc`
Phase 2 test precision / recall / F1	0.119284 / 0.483456 / 0.191354	`results/fusion_config.json`
Phase 2 test FPR	0.130014	`results/fusion_config.json` → `fusion_fpr` — see §11.1 root-cause analysis
Phase 2 confusion matrix (test, n=88,581)	TP=1,505 FP=11,112 TN=74,356 FN=1,608	`results/fusion_config.json`
Accounts profiled / avg. txns per account	13,553 accounts, 6,512 with ≥5 txns (48.0% coverage), avg 43.6 txns	`data/profile_stats.json`
Dataset size & fraud rate	590,540 transactions, 20,663 fraud (3.499%)	`data/profile_stats.json`, confirmed in NB02 output

README BADGE DISCREPANCY

README's top-of-file badges assert "AUC 96.24%" and "Latency 0.019ms" without stating which model (Phase 1 only, matches `deployment_config.json`) and separately claim, in its closing `<sub>` caption, "Account compromise ROC · AUC = 75.67%" — the actual `fusion_config.json` value is **74.06%** (`fusion_auc = 0.740583`). This 1.6-point discrepancy could not be resolved from any other artifact in the repository and is reported as-is.

1.13 Major components and their responsibilities

Component	File(s)	Responsibility
Serving API	`app.py`	Loads all 11 model artifacts once at import time; exposes 16 HTTP routes; performs the full Phase 1 + Phase 2 inference, decisioning, logging, and profile-update flow.
Shared preprocessing	`preprocessing.py`	Loads the training-time imputer/scaler; applies imputer→scaler transform; pads partial form input to the full 224-dim raw feature vector.
Behavioral profile store	`profile_store.py`	SQLite (WAL) CRUD for per-account behavioral profiles and the transaction/event audit log; alerting query; coverage diagnostics.
Kafka consumer	`profile_consumer.py`	Standalone process that subscribes to the `profile_updates` Kafka topic and logs each event (stub; does not update `ProfileStore`).
Phase 1 training	`notebooks/01-05*.ipynb`	Cleans and balances the raw IEEE-CIS CSV, trains the EARN+ extractors and AttentionRXTJ classifier, trains the IsolationForest, and runs Jaya weight/threshold optimization.
Phase 2 training	`notebooks/06-08*.ipynb`	Builds behavioral profiles and the transaction snapshot, computes the 13 contextual features and compromise labels, trains FusionNet v3.
Demo sample builder	`notebooks/save_demo_samples.py`	Extracts 8 hand-picked EARN+ feature vectors from the test split into `data/demo_samples.json`, served by `/demo-samples`.
Demo front ends	`panel.html`, `index.html`, `fraudshield_app.html`, `rxtj_app.html`	Static single-file HTML/JS dashboards that call the running API via `fetch()`; `panel.html` is also served directly by the API at `/panel`.

2. Complete File Inventory

The repository contains 44 tracked files (excluding `.git/`), enumerated below in full via `find . -type f`. No file has been omitted. Rows are grouped by directory in the order they appear on disk.

2.1 Repository root

File	Type	Purpose / role	Key classes / functions	Inputs	Outputs / consumed by
`app.py`	Python (1,151 ln)	FastAPI serving application — the entire runtime system: model loading, 16 HTTP routes, Phase 1 + Phase 2 inference, decisioning, SQLite history logging, Kafka publish.	`ResNeXtBlock`, `ResNeXtExtractor`, `SelfAttentionGRU`, `AttentionRXTJ`, `AutoencoderNet`, `ResBlock`, `FraudResNet`, `FusionNet`; `load_models`, `score_raw_features`, `extract_earn_features`, `score_earn_features`, `compute_contextual_features`, `decide_action`; 16 route functions. See §3.	11 files in `models/`, 2 in `results/`, 4 in `data/`; runtime HTTP JSON requests.	HTTP JSON responses; writes `fraud_storage.db`, `data/behavioral_profiles.db`; optional Kafka messages.
`preprocessing.py`	Python (45 ln)	Shared imputer/scaler load & transform helpers, used by `app.py` at inference time (README also claims Phase-2 profile builder uses it; NB06/07/08 in fact re-implement the same load logic inline rather than importing this module — see §11).	`load_preprocessors`, `raw_feature_dim`, `transform_raw`, `pad_form_features`	`models/imputer.pkl`, `models/scaler.pkl`; raw feature arrays.	Imputed+scaled `np.float32` arrays; consumed by `app.py`.
`profile_store.py`	Python (232 ln)	SQLite (WAL) CRUD layer for per-account behavioral profiles and the scored-event audit log.	Class `ProfileStore` (10 methods, see §3.3); module constants `_EMA_ALPHA`, `_DDL`.	Method call arguments (account_id, txn dict, etc.)	`data/behavioral_profiles.db`; consumed by `app.py` and `notebooks/06_behavioral_profiling.ipynb`.

`profile_consumer.py`	Python (77 ln)	Standalone Kafka consumer CLI for the `profile_updates` topic; currently a logging stub, not wired to `ProfileStore`.	`handle_event, main`	Kafka messages from topic `profile_updates` (env `KAFKA_PROFILE_TOPIC`).	stdout log lines only.
`README.md`	Markdown (1,033 ln / 36 KB)	Project documentation. Contains accurate high-level description but multiple stale/aspirational sections — see §11.4 for the full discrepancy list.	—	—	Human-readable only.
`requirements.txt`	pip requirements (11 ln)	Pinned Python dependency list for the serving app (does **not** cover notebook-only dependencies — see §4.3).	—	—	Consumed by `pip install -r requirements.txt`.
`.gitignore`	Git config (34 ln)	Excludes `__pycache__/`, venvs, `.ipynb_checkpoints/`, `data/IEEE CIS/`, generated `data/*.npy`, `*.db` / `*.sqlite`, `.env*`, logs.	—	—	Consumed by git.
`panel.html`	HTML/CSS/JS (953 ln)	Primary interactive demo ("RXT-J+ FraudShield — Phase 2 Demo"). Served directly by the API at `GET /panel` (app.py:592-596). Lets the user pick a scenario, choose an endpoint (`score/direct`, `score/form-full`, or `account/compromise-score`), and view the response, attention weights, and a live feed.	JS: `loadDemoSamples, checkHealth, loadScenario, updateEndpointNote, showTab, runScore, renderResult, addFeedItem` (client-side, not Python — catalogued at a functional level only).	`GET /demo-samples`, `GET /health`, and the chosen scoring endpoint.	Renders results in-page; no file output.
`index.html`	HTML/CSS/JS (198 ln)	Minimal standalone dashboard ("FraudShield | AI Risk Dashboard") calling `/history` and `/score/form` against `http://127.0.0.1:8000`.	JS: inline fetch handlers.	`GET /history`, `POST /score/form`.	Renders results in-page.
`fraudshield_app.html`	HTML/CSS/JS (251 ln)	Standalone single-form demo calling `POST /score/form`.	JS: inline fetch handler.	`POST /score/form`.	Renders results in-page.
`rxtj_app.html`	HTML/CSS/JS (730 ln)	Standalone dashboard calling `/health`, `/demo-samples`, and `/score/direct` (twice, from two different UI panels) against `http://127.0.0.1:8000`.	JS: inline fetch handlers.	`GET /health`, `GET /demo-samples`, `POST /score/direct`.	Renders results in-page.

2.2 `notebooks/`

File	Type	Purpose	Reads	Writes	Notes
`01_preprocessing.ipynb`	Notebook, 1 cell (1,118 B)	Illustrative mean-imputation + standardization stub.	`data/IEEE CIS/train_transaction.csv`	Nothing persisted (no `joblib.dump` call in this notebook — see §11).	1 code cell, never executed (no outputs, no `execution_count`). `imputer.pkl` / `scaler.pkl` are actually produced by NB02.
`02_balancing.ipynb`	Notebook, 11 cells (55 KB)	Drops high-null columns, label-encodes categoricals, imputes+scales, class-balances via RandomUnderSampler→SMOTE Tomek.	`data/IEEE CIS/train_transaction.csv`	`models/scaler.pkl`, `models/imputer.pkl`, `data/Cleaned/X_balanced.npy`, `data/Cleaned/y_balanced.npy`, `data/balance_stages.png`	Cell 0 is a duplicated/incomplete fragment of the same logic that appears fully in cells 1–10 — see §11.
`03_earn_features.ipynb`	Notebook, 11 cells (148 KB)	Trains the Autoencoder + FraudResNet extractors, fits Nystroem + IncrementalPCA ("EARN+"), validates with K-Means/silhouette.	`data/Cleaned/X_balanced.npy`, `y_balanced.npy`	`models/nystroem.pkl`, `models/incremental_pca.pkl`, `models/autoencoder.pt`, `models/resnet_extractor.pt`, `models/kernel_pca.pkl`, `data/Z_train_earn.npy`, `data/Z_test_earn.npy`, `data/y_train.npy`, `data/y_test.npy`, `data/ae_loss.png`, `data/resnet_loss.png`, `data/kpca_clusters.png`	**Not re-runnable top-to-bottom**: references `Z_train_concat` and `kpca` which are never assigned anywhere in the saved notebook — see §11.

`04_attention_rxtj.ipynb`	Notebook, 12 cells (158 KB)	Defines and trains `AttentionRXTJ` (ResNeXt + SelfAttentionGRU) on the 50-dim EARN+ features; trains `IsolationForest`; computes the Phase-1 combined risk score.	`data/Z_train_earn.npy`, `Z_test_earn.npy`, `y_train.npy`, `y_test.npy`	`models/attention_rxtj.pt`, `models/isolation_forest.pkl`, `data/risk_scores.npy`, `data/model_probs.npy`, `data/ifm_scores.npy`, `data/training_curves.png`, `data/roc_curve.png`, `data/confusion_matrix.png`	Fully executed with outputs; Best Val AUC 0.9682.
`05_jaya_optimize.ipynb`	Notebook, 11 cells (200 KB)	Jaya metaheuristic search over (W_MODEL, W_IFM); threshold sweep; final evaluation; latency/throughput benchmark.	`data/model_probs.npy`, `data/ifm_scores.npy`, `data/y_test.npy`; (benchmark cell also loads `models/nystroem.pkl`, `incremental_pca.pkl`, `isolation_forest.pkl`)	`results/deployment_config.json`, `models/jaya_optimal_weights.pkl`, `results/jaya_convergence.png`, `results/roc_comparison.png`, `results/confusion_matrix_final.png`, `results/threshold_optimisation.png`	Latency benchmark (Cell 9) times only Nystroem+IPCA+IFM on random noise, not the model or AE/ResNet stages — see §11.1.
`06_behavioral_profiling.ipynb`	Notebook, 11 cells (17 KB)	Builds per-account SQLite profiles via `ProfileStore`; computes rolling amount stats, 1h/24h velocity, device novelty; writes the transaction snapshot.	`data/IEEE CIS/train_transaction.csv`, `train_identity.csv`	`data/behavioral_profiles.db`, `data/tx_snapshot.parquet`, `data/profile_stats.json`	Implements the "smart resume" logic described in the header docstring — see §10.6.
`07_contextual_features.ipynb`	Notebook, 15 cells (28 KB)	Computes the 13 contextual features and the 24h-window account-compromise label.	`data/tx_snapshot.parquet`, `models/imputer.pkl`, `scaler.pkl`, `autoencoder.pt`	`data/contextual_features.npy`, `data/fusion_labels.npy`, `data/fusion_account_ids.npy`, `data/fusion_feature_names.json`, `data/drift_norm_params.json`, `data/merchant_fraud_rates.json`	Feature 7 (`p1_risk_score`) is saved as a constant placeholder `0.5` for all rows — patched later by NB08. Emits pandas `FutureWarning`s on two `groupby().apply()` calls — see §11.2.

`08_fusion_model_training.ipynb`	Notebook, 2 cells (41 KB)	Regenerates real Phase-1 scores for the placeholder column, then trains FusionNet v3 with Focal Loss on a temporal split, then Jaya-optimizes the decision threshold.	`data/contextual_features.npy`, `fusion_labels.npy`, `fusion_account_ids.npy`, `fusion_feature_names.json`, `tx_snapshot.parquet`, all Phase-1 `models/*`, `data/IEEE CIS/train_transaction.csv`	`models/fusion_net.pt`, `models/fusion_scaler.pkl`, `results/fusion_config.json`, `results/fusion_roc.png`, `results/fusion_training_curves.png`	Self-documented shortfall: the notebook's own printed targets (AUC≥0.85, MCC≥0.35, Precision≥0.15, FPR≤0.10) are all missed by the achieved result — see §11.1 (root-caused, not hypothesis).
`save_demo_samples.py`	Python (42 ln)	One-off script producing the 8 demo feature vectors served by `/demo-samples`.	`data/Z_test_earn.npy`, `data/y_test.npy`	`data/demo_samples.json`	Run manually from inside `notebooks/` ("Run once in Jupyter or VS Code"); not part of the numbered pipeline.

2.3 `models/` (11 files, all binary, all consumed by `app.py` except two)

File	Size	Format / type	Produced by	Consumed by
`imputer.pkl`	3,975 B	joblib — `sklearn.impute.SimpleImputer`	NB02 Cell 10	`preprocessing.load_preprocessors` → `app.py`; NB07, NB08 (reloaded directly, not via `preprocessing.py`)
`scaler.pkl`	5,991 B	joblib — `sklearn.preprocessing.StandardScaler`	NB02 Cell 10	same as above
`nystroem.pkl`	515,468 B	joblib — `sklearn.kernel_approximation.Nystroem`	NB03 Cell 8	`app.py extract_earn_features`; NB05 benchmark; NB08 Stage 1
`incremental_pca.pkl`	126,963 B	joblib — `sklearn.decomposition.IncrementalPCA`	NB03 Cell 8	same as above
`autoencoder.pt`	813,595 B	PyTorch state_dict — `AutoencoderNet` / `FraudAutoencoder`	NB03 Cell 10	`app.py` (EARN+ extraction + behavioral drift); NB07, NB08
`resnet_extractor.pt`	428,301 B	PyTorch state_dict — `FraudResNet`	NB03 Cell 10	`app.py` (EARN+ extraction); NB08

`kernel_pca.pkl`	5,729,216 B	joblib — `sklearn.decomposition.KernelPCA`	NB03 Cell 10 (`joblib.dump(kpca, ...)`)	*Orphaned — never loaded by `app.py` or any other file. The variable `kpca` that is dumped is itself never assigned inside the saved notebook (see §11). Superseded by the Nystroem+IncrementalPCA path.*
`attention_rxtj.pt`	511,277 B	PyTorch state_dict — `AttentionRXTJ` (244 tensor keys)	NB04 Cell 11	`app.py score_earn_features` ; NB05 (implicitly, via saved scores); NB08 Stage 1
`isolation_forest.pkl`	1,077,849 B	joblib — `sklearn.ensemble.IsolationForest`	NB04 Cell 11	`app.py score_earn_features` ; NB05; NB08 Stage 1
`jaya_optimal_weights.pkl`	241 B	joblib — raw `numpy.ndarray` `[1.0, 0.87162169]`	NB05 Cell 9	*Orphaned — `app.py` reads the already-normalized W_MODEL/W_IFM from `deployment_config.json` instead of this file. Referenced nowhere outside NB05 itself.*
`fusion_net.pt`	52,598 B	PyTorch state_dict — `FusionNet` (9 tensor keys)	NB08	`app.py score_form_full` , `account_compromise_score`
`fusion_scaler.pkl`	911 B	joblib — `sklearn.preprocessing.StandardScaler` (13 features)	NB08	`app.py` (scales the 13-dim contextual vector before FusionNet)

2.4 `data/`

File	Type	Purpose / schema	Producer	Consumer
`demo_samples.json`	JSON (9,322 B)	Dict of 8 named 50-dim EARN+ feature vectors (`fraud_1..3, legit_1..3, edge_1..2`).	`notebooks/save_demo_samples.py`	`app.py GET /demo-samples` ; `panel.html` , `rxtj_app.html`
`drift_norm_params.json`	JSON (54 B)	`{"p5": 0.2056, "p95": 0.2463}` — 5th/95th percentile of AE reconstruction MSE, used to normalize the `behavioral_drift` feature to [0,1].	NB07 Cell 4	`app.py compute_contextual_features` (feature F7)
`fusion_feature_names.json`	JSON (284 B)	Ordered list of the 13 contextual feature names.	NB07 Cell 14	Documentation cross-check only — `app.py` instead loads the (identical) list from `results/fusion_config.json["feature_names"]` .
`merchant_fraud_rates.json`	JSON (87 B)	`{C:0.1169, H:0.0477, R:0.0378, S:0.0590, W:0.0204}` — per-ProductCD historical fraud rate.	NB07 Cell 9	`app.py compute_contextual_features` (feature F11)
`profile_stats.json`	JSON (358 B)	Coverage diagnostics: total transactions, unique accounts, fraud rate, profile coverage, snapshot path, generation timestamp.	NB06 Cell 9	Documentation / manual inspection only; not read by `app.py` .

`tx_snapshot.parquet`	Parquet (19,832,680 B)	590,540-row per-transaction snapshot; 15 columns (schema in §9.4).	NB06 Cell 8	NB07, NB08 (temporal split, snapshot re-reads)
`ae_loss.png`	PNG (19,954 B)	Autoencoder training-loss curve.	NB03	Documentation / README figure reference.
`balance_stages.png`	PNG (30,735 B)	Class-balance bar chart at 3 pipeline stages.	NB02	Documentation.
`confusion_matrix.png`	PNG (14,807 B)	AttentionRXTJ confusion matrix (pre-Jaya).	NB04	Documentation.
`kpca_clusters.png`	PNG (53,884 B)	K-Means/true-label scatter over first 2 EARN+ dims.	NB03	Documentation.
`resnet_loss.png`	PNG (18,636 B)	FraudResNet training-loss curve.	NB03	Documentation.
`roc_curve.png`	PNG (29,060 B)	AttentionRXTJ ROC curve (pre-Jaya).	NB04	Documentation.
`training_curves.png`	PNG (56,369 B)	AttentionRXTJ loss + validation-AUC curves.	NB04	Documentation; README image reference.

2.5 `results/`

File	Type	Purpose / schema	Producer	Consumer
`deployment_config.json`	JSON (444 B)	Phase-1 deployment config: Jaya weights, threshold, evaluation metrics, latency/throughput. Full key table in §7.1.	NB05 Cell 9	`app.py` `load_models` (`W_MODEL`, `W_IFM`, `THRESHOLD`); `/`, `/model/info`
`fusion_config.json`	JSON (1,212 B)	Phase-2 deployment config: thresholds, 13 feature names, learned attention weights, evaluation metrics. Full key table in §7.2.	NB08	`app.py` `load_models` (thresholds, `FEATURE_NAMES`); `/model/info`
`confusion_matrix_final.png`	PNG (16,239 B)	Phase-1 confusion matrix after Jaya threshold optimization.	NB05	Documentation.
`fusion_roc.png`	PNG (135,613 B)	3-panel figure: FusionNet ROC, precision-recall curve, attention-weight bar chart.	NB08	Documentation; README image reference.
`fusion_training_curves.png`	PNG (27,012 B)	FusionNet training loss + validation-AUC curves.	NB08	Documentation.
`jaya_convergence.png`	PNG (27,057 B)	Jaya objective-value convergence curve (Phase 1).	NB05	Documentation.
`roc_comparison.png`	PNG (40,569 B)	Baseline vs. Jaya-optimized ROC overlay (Phase 1).	NB05	Documentation.

`threshold_optimisation.png`	PNG (45,536 B)	Fiscal-cost / MCC vs. threshold sweep (Phase 1).	NB05	Documentation.

2.6 Files described in README.md but absent from the repository

Path claimed by README	Status
`tests/` (and all 4 files inside)	*Not present in repository.*
`.env.example`	*Not present in repository.*
`Dockerfile`	*Not present in repository.*
`docker-compose.yml`	*Not present in repository.*
`.github/workflows/deploy.yml`	*Not present in repository.*
`docs/assets/` (logo.png, demo.gif, architecture.png, roc.png)	*Not present in repository.*
`fix_p1_scores.py` (referenced in README "Troubleshooting")	*Not present in repository.*
`data/IEEE CIS/train_transaction.csv`, `train_identity.csv`	*Not present in repository (excluded by `.gitignore`; must be downloaded from Kaggle per README instructions). Required to re-run notebooks 01, 02, 06, 08 from scratch.*
`data/behavioral_profiles.db`, `fraud_storage.db`	*Not present in repository (gitignored, runtime-generated — see §9).*

3. Complete Class & Function Catalog

Extracted by parsing the Python AST of every `.py` file in the repository (`app.py`, `preprocessing.py`, `profile_store.py`, `profile_consumer.py`, `notebooks/save_demo_samples.py`). Line numbers are exact. `FocalLoss` is defined only inside `notebooks/08_fusion_model_training.ipynb` (not a `.py` file) and is catalogued in §3.6 from the notebook source directly.

3.1 app.py — model classes (PyTorch `nn.Module`)

Class	Line	Constructor signature	Purpose
`ResNeXtBlock`	45	`__init__(self, in_dim, out_dim, cardinality=CARDINALITY)`	One ResNeXt "grouped-linear" residual block: `cardinality` (=4) parallel 2-layer MLP paths of width `out_dim/cardinality`, concatenated, summed with a linear/identity shortcut, ReLU.
`ResNeXtExtractor`	61	`__init__(self, input_dim)`	Stacks 4 `ResNeXtBlock`s: `input_dim→128→128→64→64`. Produces the 64-dim transaction embedding used by `SelfAttentionGRU`.
`SelfAttentionGRU`	70	`__init__(self, input_dim=64, hidden_dim=64, seq_len=SEQ_LEN)`	Reshapes the 64-dim embedding into `(seq_len=8, step_dim=8)`, runs a 2-layer GRU (dropout 0.3), computes a softmax attention weight per timestep via a `Linear(64,32)→Tanh→Linear(32,1)` head, context-pools, and classifies via `Linear(64,32)→ReLU→Dropout(0.3)→Linear(32,1)→Identity` (raw logit; sigmoid applied by the caller).
`AttentionRXTJ`	88	`__init__(self, input_dim, seq_len=SEQ_LEN)`	Composes `ResNeXtExtractor(input_dim)` → `SelfAttentionGRU`. This is the Phase-1 classifier; the trained weights live in `models/attention_rxtj.pt`.
`AutoencoderNet`	96	`__init__(self, input_dim, latent_dim=64)`	Encoder `input_dim→256→128→latent_dim` (BatchNorm+ReLU+Dropout(0.2) per hidden layer); decoder mirrors back to `input_dim`. Returns `(reconstruction, latent)`.
`ResBlock`	114	`__init__(self, in_dim, out_dim)`	Docstring: "Residual block used by FraudResNet (must match resnet_extractor.pt weights)." Two-layer `Linear→BatchNorm` block + linear/identity shortcut + ReLU.
`FraudResNet`	127	`__init__(self, input_dim, out_dim=64)`	Docstring: "Feature extractor trained in notebook 03 (resnet_extractor.pt). `.features()` returns a 64-dim representation from 224-dim scaled input." `stem: Linear(input_dim,128)` → 4× `ResBlock` (128→128→64→64→out_dim) → `.features()`; `head: Linear(out_dim,2)` for the original training-time 2-class objective (unused at inference — only `.features()` is called).

`FusionNet`	153	`__init__(self, input_dim: int = 13)`	Docstring: "FusionNet v3 — Attention-weighted MLP for account compromise scoring. Architecture is fixed to match the saved weights in fusion_net.pt. Input dim MUST be 13." Learnable `feature_attn` parameter (softmax-gated element-wise feature reweighting) feeding `Linear(13,128)→ReLU→Dropout(0.3)→Linear(128,64)→ReLU→Dropout(0.3)→Linear(64,32)→ReLU→Linear(32,1)`; returns `(sigmoid(logit), attn_weights)`.

> **SOURCE-DOCUMENTED HISTORY**
>
> The file header of `app.py` (lines 5–13) records three specific fixes applied to reach the current architecture: **[BUG-1]** the `FusionNet` tail was corrected from an earlier 32→16→1 layout (which caused a state_dict shape mismatch against `fusion_net.pt`) to the current 32→1; **[BUG-2]** `compute_contextual_features` was expanded to produce all 13 features (5 were previously missing); **[BUG-3]** the `/account/explain` `interpret()` lookup was updated for all 13 feature names.

3.2 app.py — module-level functions

Function	Line	Signature	Purpose / side effects
`load_models`	185	`load_models()`	Loads all 11 model artifacts + 2 JSON configs from disk (file I/O); constructs and `load_state_dict`s all 4 PyTorch modules; returns a 12-tuple. Called once at import time (line 229) — not re-callable without restarting the process.
`_init_history_db`	257	`_init_history_db()`	`CREATE TABLE IF NOT EXISTS history` in `fraud_storage.db` (DB I/O, commits, closes). Called once at import time (line 267).
`_publish_profile_update`	297	`_publish_profile_update(transaction_id, risk_score, decision, features)`	Fire-and-forget Kafka publish to `KAFKA_TOPIC`; no-ops if `kafka_producer is None`; swallows all exceptions (bare `except Exception: pass`) — network I/O, non-blocking, no error surfacing. Called from `score_single` and `score_form` only (not from the Phase-2 endpoints).
`score_earn_features`	388	`score_earn_features(Z_earn: np.ndarray)`	Docstring: "Score 50-dim EARN+ features directly — no Nystroem/IPCA needed." Runs `model(Z_earn)` → sigmoid; runs `ifm.score_samples(Z_earn)`, min-max normalizes and inverts (so higher = more anomalous); fuses via `(W_MODEL/total)·probs + (W_IFM/total)·ifm_norm`; thresholds at `THRESHOLD`. Returns `(risk, preds, conf)`. Called by `score_direct`, `score_raw_features`.
`extract_earn_features`	406	`extract_earn_features(X_clean: np.ndarray) -> np.ndarray`	Docstring documents the exact 5-step pipeline (AE encoder + ResNet extractor → concat(128) → Nystroem(300) → IncrementalPCA(50)). Two `torch.no_grad()` forward passes + two sklearn `.transform()` calls. Called by `score_raw_features`.

`score_raw_features`	435	`score_raw_features(features_np: np.ndarray)`	Composition: `transform_raw` → `extract_earn_features` → `score_earn_features`. This is the single entry point used by every Phase-1-from-raw endpoint (`/score` , `/score/form` , `/score/batch` , `/score/form-full`).
`compute_contextual_features`	445	`compute_contextual_features(features_np: np.ndarray, profile: dict, txn_meta: dict) -> tuple`	The 13-feature Phase-2 feature engineering function. Pure computation except one `torch.no_grad()` autoencoder forward pass for F7. Full formula breakdown in §5.3 and §14. Returns `(vec: np.ndarray shape (1,13), named: dict)` .
`decide_action`	584	`decide_action(compromise_prob: float, p1_risk: float) -> str`	Docstring: "Map compromise probability + P1 risk to a recommended action." Pure function, no side effects. Full decision table in §14.
`interpret` (nested in `explain_compromise`)	1094	`interpret(name, val)`	Docstring: "Human-readable interpretation for all 13 feature names." Static dict lookup producing an English sentence per feature; falls back to `"{name} = {val:.4f}"` for unrecognized names.

3.3 app.py — API endpoint functions

All 16 route handlers, in source order. Full request/response schemas are in §8.

Line	Function	Route	Behavior summary
593	`serve_panel`	`GET /panel`	Returns `panel.html` as a `FileResponse` .
601	`root`	`GET /`	Returns service banner + 4 headline metrics read from `config` / `fusion_config` .
614	`health`	`GET /health`	Returns a static "healthy" status + `fusion_input_dim` + `feature_count` . Does not actually probe DB/model liveness.
623	`model_info`	`GET /model/info`	Returns nested Phase-1 and Phase-2 metadata (weights, thresholds, performance, speed) sourced entirely from the two loaded config JSONs.
658	`demo_samples`	`GET /demo-samples`	Streams `data/demo_samples.json` ; 404 if missing.
669	`score_single`	`POST /score`	Full Phase-1 scoring of a raw 224-dim vector; publishes to Kafka; no DB write.
690	`score_form`	`POST /score/form`	Pads a 3-field partial input to 224-dim, scores, **writes to `fraud_storage.db.history`** , publishes to Kafka.
720	`get_history`	`GET /history`	Reads the last *N* (≤200) rows of `fraud_storage.db.history` .
730	`score_direct`	`POST /score/direct`	Scores a pre-computed 50-dim EARN+ vector directly (skips imputer/scaler/AE/ResNet/Nystroem/IPCA); validates dimension against `ipca.n_components_` .
757	`score_batch`	`POST /score/batch`	Batch-scores up to 1,000 raw 224-dim vectors in one `score_raw_features` call; returns per-row results + aggregate counts.

787	`score_form_full`	`POST /score/form-full`	Full 9-step Phase 1 + Phase 2 pipeline from human-readable form fields; writes to `ProfileStore` (log + profile update). Documented step-by-step in §5.3.
925	`account_compromise_score`	`POST /account/compromise-score`	Same Phase 2 pipeline as above but accepts either a raw 224-dim or pre-computed 50-dim feature vector; writes to `ProfileStore`.
1040	`get_account_profile`	`GET /account/profile/{account_id}`	Reads one profile row; 404 if unknown; adds computed `profile_age_hours`, `known_device_count`.
1056	`get_account_history`	`GET /account/history/{account_id}`	Reads last *N* (≤100) `transaction_log` rows for the account.
1068	`get_account_alerts`	`GET /account/alerts`	Aggregate query: accounts with `MAX(compromise_prob) ≥ threshold` in the last 24h.
1081	`explain_compromise`	`POST /account/explain`	Looks up a previously-logged event by `(account_id, transaction_id)`; 404 if not found; builds a natural-language explanation from stored context + `interpret()`.

3.4 app.py — Pydantic request/response models

Model	Line	Fields
`TransactionRequest`	329	`transaction_id: str` (example "TXN_001"); `features: List[float]` (description: "Preprocessed feature vector")
`DirectScoreRequest`	333	`transaction_id: str`; `features: List[float]` (description: "50-dim EARN+ feature vector (post-IPCA)")
`BatchRequest`	337	`transactions: List[TransactionRequest]`
`FormScoreRequest`	340	`transaction_id: str`; `amount: float`; `is_foreign: bool = False`; `customer_age: Optional[int] = 30`
`RiskResponse`	346	`transaction_id: str`; `risk_score: float`; `decision: str`; `confidence: float`; `latency_ms: float`
`CompromiseRequest`	353	`account_id: str`; `transaction_id: str`; `features: List[float]`; `amount: Optional[float] = 0.0`; `hour: Optional[int] = 0`; `product_cd: Optional[str] = ""`; `device_info: Optional[str] = None`; `addr1: Optional[str] = None`; `timestamp: Optional[float] = None`
`ExplainRequest`	364	`account_id: str`; `transaction_id: str`
`FullFormRequest`	368	`account_id: str` (ex. "ACC_42"); `transaction_id: str` (ex. "TXN_001"); `amount: float` (ex. 289.99); `hour: int = 14` (ge=0, le=23); `product_cd: str = "W"`; `is_foreign: bool = False`; `customer_age: Optional[int] = 30` (ge=0, le=120); `device_info: Optional[str] = None`; `addr1: Optional[str] = None`; `timestamp: Optional[float] = None`

3.5 preprocessing.py — module functions

Function	Line	Signature	Purpose
`load_preprocessors`	12	`load_preprocessors(base_dir)`	Loads `models/imputer.pkl` and `models/scaler.pkl` via `joblib.load`; returns `(imputer, scaler)`. File I/O only.
`raw_feature_dim`	19	`raw_feature_dim(imputer)`	Returns `int(imputer.statistics_.shape[0])` — 224 in practice. Pure.
`transform_raw`	24	`transform_raw(features, imputer, scaler)`	Coerces to a 2-D `float32` array, applies `imputer.transform` then `scaler.transform`. Pure (no I/O beyond the passed-in fitted objects).
`pad_form_features`	38	`pad_form_features(values, target_dim)`	Allocates a `target_dim`-length `NaN` array, fills the first `len(values)` positions with `values`, leaves the rest `NaN` so the downstream imputer substitutes training-set column means. Pure.

3.6 profile_store.py — `ProfileStore` class (all methods)

Method	Line	Signature	Behavior / I/O
`__init__`	59	`__init__(self, db_path: str = "data/behavioral_profiles.db")`	Creates the parent directory if needed; opens a connection and runs `_DDL` (creates `profiles`, `transaction_log` tables + 2 indexes, sets WAL mode + `synchronous=NORMAL`). DB I/O, state mutation (file creation).
`_conn`	65	`_conn(self)`	Opens a fresh `sqlite3.connect(db_path, timeout=10)` with `row_factory=sqlite3.Row`. Every public method calls this and lets it close via a `with` block — one connection per call, no persistent pool.
`_parse`	73	`@staticmethod _parse(row) -> dict`	Converts a `sqlite3.Row` to a dict and `json.loads`-decodes the three JSON-text columns (`hour_hist`, `merchant_counts`, `known_devices`). Pure.
`get_profile`	82	`get_profile(self, account_id: str) -> Optional[dict]`	`SELECT * FROM profiles WHERE account_id=?`; returns `None` if not found. DB read.
`get_or_empty`	90	`get_or_empty(self, account_id: str) -> dict`	Calls `get_profile`; on miss returns a hand-coded zero-state profile dict (uniform 24-slot hour histogram, empty merchant/device collections). DB read.

`update_profile`	106	`update_profile(self, account_id: str, txn: dict) -> None`	EMA-blends amount mean/std (α=0.1, both overall and a separate 7-day-labeled EMA that is *not actually time-windowed* — see §11), updates the 24-bin hour histogram via renormalized EMA, increments merchant/product-code counts, appends new devices (capped at the 50 most recent), updates `geo_cluster` = `addr1` if provided. `INSERT` on first-seen account, else `UPDATE`. DB write, state mutation. **Does not update `velocity_1h` / `velocity_24h` columns** — they remain at their `INSERT`-time defaults (0) forever for accounts created via the live API (see §11).
`log_transaction`	165	`log_transaction(self, account_id, transaction_id, p1_risk_score, decision, timestamp=None, compromise_prob=None, top_trigger="", context=None) -> None`	`INSERT` into `transaction_log`, JSON-encoding `context`. DB write.
`get_history`	183	`get_history(self, account_id: str, limit: int = 20) -> list`	`SELECT ... FROM transaction_log WHERE account_id=? ORDER BY id DESC LIMIT ?` (limit clamped to [1,200]); JSON-decodes `context_json` into `context`. DB read.
`get_alerts`	199	`get_alerts(self, threshold: float = 0.7, limit: int = 50) -> list`	Aggregate query grouping by `account_id` where `compromise_prob≥threshold` and `timestamp≥now-86400`, ordered by `MAX(compromise_prob)` desc (limit clamped to [1,500]). Nested helper `_action(s)` (line 212) maps the max score to a 3-tier recommended action. DB read.
`coverage_stats`	224	`coverage_stats(self) -> dict`	3 aggregate `SELECT COUNT` / `AVG` queries → `{total_accounts, accounts_5plus, coverage_pct, avg_txn_count}`. DB read.

3.7 profile_consumer.py — module functions

Function	Line	Signature	Behavior
`handle_event`	23	`handle_event(event)`	Docstring: "Override / extend this with the Phase 2 profile-store update logic. For now we just log to stdout so the wiring can be verified end-to-end." Prints `tid`, `decision`, `risk` from the event dict. **Does not call `ProfileStore`**.
`main`	32	`main()`	Imports `kafka.KafkaConsumer` lazily (exits with code 1 if not installed); connects with `auto_offset_reset='earliest'`, `enable_auto_commit=True`, JSON value deserializer; installs `SIGINT` / `SIGTERM` handlers via nested `_shutdown`; iterates the consumer forever, calling `handle_event` per message with a 0.1s sleep-and-continue on handler exceptions.
`_shutdown`	56	`_shutdown(*_)` (nested in `main`)	Signal handler: closes the consumer and calls `sys.exit(0)`.

3.8 notebooks/save_demo_samples.py

Script-level code, no function definitions. Loads `data/Z_test_earn.npy` / `y_test.npy`, selects 8 fixed indices (fraud indices 0, 10, 50, 200; legit indices 0, 10, 50, 200) into a dict with keys `fraud_1..3, legit_1..3, edge_1, edge_2`, and writes `data/demo_samples.json`.

3.9 Notebook-defined classes not present in app.py (training-time only)

Class / function	Notebook · location	Purpose & notable difference from the `app.py` counterpart
`FraudAutoencoder`	NB03 Cell 2; re-declared in NB07 as `_AE`, in NB08 as `FraudAutoencoder` with an added `.encode()` method	Identical architecture to `app.py`'s `AutoencoderNet`; different class name only.
`ResBlock` / `FraudResNet`	NB03 Cell 5; re-declared in NB08 with class names prefixed `_ResBlock`	Identical architecture to `app.py`. NB08's copy adds a `.extract()` alias for `.features()`.
`ResNeXtBlock` / `ResNeXtExtractor`	NB04 Cells 2–3; re-declared in NB08 as `_ResNeXtBlock` / `_ResNeXtExtractor`	Identical to `app.py`, except the NB04 version has an explicit `assert out_dim % cardinality == 0` sanity check that `app.py`'s copy omits.
`SelfAttentionGRU`	NB04 Cell 3; NB08 as `_SelfAttentionGRU`	NB04's **training-time** classifier head ends in `nn.Sigmoid()` (returns probabilities); this final layer is monkey-patched to `nn.Identity()` in Cell 7 (`model.attn_gru.classifier[-1] = nn.Identity()`) before training with `BCEWithLogitsLoss`, matching `app.py`'s permanent `nn.Identity()` definition (a parameter-free module, so the saved `state_dict` is identical either way).
`AttentionRXTJ`	NB04 Cell 4; NB08	Identical composition to `app.py`.
`FusionNet`	NB08 only	Matches `app.py`'s FusionNet v3 exactly (13→128→64→32→1, `feature_attn` softmax gate); NB08 additionally defines a free function `_logits(m, x)` that returns the pre-sigmoid logit for use with `FocalLoss`.
`FocalLoss`	NB08, defined once	`__init__(self, alpha=0.75, gamma=2.0)`; `forward(self, logits, targets)` implements Lin et al. 2017 focal loss: `bce = BCEWithLogits(logits,targets,reduction='none'); p_t = p·y+(1-p)·(1-y); alpha_t = α·y+(1-α)·(1-y); loss = alpha_t·(1-p_t)^γ·bce`, mean-reduced. Used as the sole training criterion for FusionNet v3. **Not present anywhere in `app.py`** — it is a training-only construct; inference uses the trained weights directly.
`jaya_optimize`	NB05 Cell 4	`jaya_optimize(obj_fn, bounds, n_pop=30, n_iter=100, seed=42)` — full Jaya metaheuristic implementation (population init, per-iteration pull-toward-best/push-from-worst update, clip-to-bounds, greedy replacement). See §5.1 for the exact update formula.

`rxtj_plus_score`	NB05 Cell 11	A packaged "production-ready" scorer function documented as reference for a deployment layer; `app.py`'s `score_raw_features` / `score_earn_features` implement the same logic independently rather than importing this notebook cell.

4. Library & Tool Inventory

4.1 requirements.txt (verbatim)

```
fastapi==0.110.0
uvicorn==0.29.0
pydantic==2.6.4
numpy==1.26.4
torch==2.2.2
scikit-learn==1.4.2
joblib==1.3.2
kafka-python==2.0.2
pandas==2.2.2
pyarrow==16.0.0
matplotlib==3.8.4
```

README VS REQUIREMENTS.TXT

README's own "Setup & Installation" section prints a *different* requirements block (`pandas>=2.0.0` , `pyarrow>=14.0.0` , `matplotlib>=3.7.0` , unpinned) than the actual `requirements.txt` file on disk (exact pins shown above). The actual file is authoritative for installation.

4.2 Third-party dependency table

Library	Version	Purpose (as actually used)	Where used
PyTorch (`torch` , `torch.nn`)	2.2.2	All four neural-network models: `AttentionRXTJ` (ResNeXt+GRU), `AutoencoderNet` , `FraudResNet` , `FusionNet` . `torch.no_grad()` inference, `state_dict` load, `torch.FloatTensor` , `torch.softmax` / `sigmoid` / `tanh` .	`app.py` (imports L25-26); NB03, NB04, NB05 (benchmark only), NB07, NB08.
FastAPI	0.110.0	REST framework: `FastAPI` , `HTTPException` , `CORSMiddleware` , `JSONResponse` , `FileResponse` , `StaticFiles` (imported, unused — see §11).	`app.py` L16-20.
Pydantic	2.6.4	Request/response schema validation: `BaseModel` , `Field` (with `example=` , `description=` , `ge=` / `le=` constraints). 8 model classes (§3.4).	`app.py` L21.
Uvicorn	0.29.0	ASGI server that runs `app.py` 's FastAPI instance (`uvicorn app:app`). Not imported inside any `.py` file — invoked only from the command line.	README run instructions; not imported in source.

NumPy (`numpy`)	1.26.4	Array construction/reshaping (`np.array`, `.reshape`), clipping (`np.clip`), statistics (`.mean/.std/.min/.max`), `np.where`, `np.full`, `np.concatenate`.	Every Python source file and every notebook.
scikit-learn (`sklearn`)	1.4.2	`SimpleImputer`, `StandardScaler`, `LabelEncoder` (preprocessing); `Nystroem`, `IncrementalPCA`, `KernelPCA` (dimensionality reduction); `IsolationForest` (anomaly scoring); `KMeans`, `silhouette_score`, `adjusted_rand_score` (cluster validation, NB03 only); `roc_auc_score`, `matthews_corrcoef`, `precision_score`, `recall_score`, `f1_score`, `roc_curve`, `confusion_matrix`, `ConfusionMatrixDisplay`, `precision_recall_curve`, `average_precision_score` (evaluation).	`app.py`, `preprocessing.py`; NB01–NB05, NB08.
joblib	1.3.2	Serialization/deserialization of every non-PyTorch model artifact (`joblib.dump` / `joblib.load`).	`app.py`, `preprocessing.py`; NB02, NB03, NB04, NB05, NB07, NB08.
kafka-python (`kafka`)	2.0.2	`KafkaProducer` (optional, lazily imported only if `KAFKA_BOOTSTRAP_SERVERS` is set); `KafkaConsumer` (lazily imported inside `main()`, process exits if missing).	`app.py` L282; `profile_consumer.py` L34.
pandas	2.2.2	CSV/Parquet loading, `groupby` / `expanding` / `rolling` feature engineering, `DataFrame.merge`, category encoding.	NB01, NB02, NB06, NB07, NB08 (not used in `app.py` or any other `.py` file).
pyarrow	16.0.0	Parquet read/write backend for `pandas.read_parquet` / `to_parquet`.	NB06 (write), NB07/NB08 (read); not imported directly (used implicitly via pandas).
matplotlib	3.8.4	All training/evaluation figures (loss curves, ROC, confusion matrix, convergence plots, attention-weight bar charts).	NB02–NB05, NB08. Not used in `app.py`.
imbalanced-learn (`imblearn`)	*Not pinned — version not determinable from repository*	`SMOTETomek`, `RandomUnderSampler`, `SMOTE` for class balancing.	NB01 (stub cell only), NB02. **Used but absent from `requirements.txt`** — see §11.

4.3 Standard-library modules used

Module	Used in	Purpose
`os`	All Python files, all notebooks	Path joining, directory creation, environment variable access.
`sys`	`app.py`, `profile_consumer.py`, NB06, NB07	`sys.path` manipulation to import sibling modules; `sys.exit`, `sys.stderr`.

`sqlite3`	`app.py`, `profile_store.py`	Direct SQLite connections (history DB in `app.py`; profile/log DB in `profile_store.py`).
`json`	`app.py`, `profile_store.py`, `profile_consumer.py`, `save_demo_samples.py`, NB05–NB08	Config loading, JSON-text column encode/decode, Kafka message (de)serialization.
`time`	`app.py`, `profile_store.py`, `profile_consumer.py`, NB05, NB06, NB08	Latency measurement, timestamps.
`typing` (`List`, `Optional`)	`app.py`, `profile_store.py`	Pydantic field typing; method signature typing.
`signal`	`profile_consumer.py`	`SIGINT` / `SIGTERM` graceful-shutdown handlers.
`gc`	NB03, NB04	Explicit `gc.collect()` calls to free RAM between heavy array-allocation stages (documented as a deliberate OOM-avoidance measure — see §5.1).
`warnings`	NB02–NB08	`warnings.filterwarnings('ignore')` — blanket-suppresses all warnings including the pandas `FutureWarning`s noted in §11.2.
`collections.Counter`	NB02	Class-balance reporting.
`pathlib.Path`	NB08	`Path.cwd().parent` for root-relative path resolution.

4.4 Tooling referenced but not present as repository files

Tool	README claim	Repository reality
pytest	Full "Testing" section with example test code	*No `tests/` directory, no test files, no `pytest` entry in `requirements.txt`.*
Docker / docker-compose	Full Dockerfile + docker-compose.yml listings	*Neither file exists.*
GitHub Actions	Full `.github/workflows/deploy.yml` listing	*No `.github/` directory.*
gunicorn	Production deployment command shown	*Not in `requirements.txt`; install instruction is `pip install gunicorn` at deploy time.*
black	"Use `black` for formatting" (Contributing section)	*Not in `requirements.txt`; no formatting config file present.*
Conda / Anaconda	Primary documented environment manager	*No `environment.yml` in the repository; notebook kernel metadata instead shows a mix of conda env names (`rxtj`, `rxt_env`) and a plain `.venv`, indicating the notebooks were authored across at least two different local environments (see §10).*

5. Complete Data Flow & Workflows

5.1 Phase 1 Training Pipeline (notebooks 01–05)

Verified step-by-step against the executed cell outputs in each notebook (exact shapes and metrics below are the actual printed values, not estimates).

Step	Notebook·cell	Input → shape	Transformation / algorithm / parameters	Output → shape	Artifact
1. Load + clean	NB02 Cells 2–3	`train_transaction.csv` → (590540, 394)	Drop columns with >70% nulls (168 dropped).	(590540, 226)	in-memory only
2. Encode categoricals	NB02 Cell 4	(590540, 226)	`LabelEncoder` on 13 object columns (`ProductCD, card4, card6, P_emaildomain, M1–M9`), NaN→"MISSING" first.	(590540, 226), all-numeric	in-memory only
3. Split X/y	NB02 Cell 5	(590540, 226)	Drop `isFraud` , `TransactionID` .	X: (590540, 224), y: (590540,)	in-memory only
4. Impute + scale	NB02 Cell 6	(590540, 224)	`SimpleImputer(strategy='mean')` → `StandardScaler()` .	(590540, 224)	**`models/imputer.pkl`** , **`models/scaler.pkl`** (saved at Cell 10, not here — fit objects held in memory until then)
5. Undersample	NB02 Cell 7	(590540, 224), 20,663 fraud / 569,877 legit	`RandomUnderSampler(sampling_strategy={0: fraud_count*10, 1: fraud_count}, random_state=42)` — keeps legit:fraud at 10:1 before the slow step.	(227293, 224), 206,630 legit / 20,663 fraud	in-memory only
6. Balance	NB02 Cell 8	(227293, 224)	`SMOTETomek(smote=SMOTE(k_neighbors=5, random_state=42), random_state=42)` .	(412416, 224), 206,208 / 206,208 (exact 1.000 ratio)	`data/Cleaned/X_balanced.npy` , `y_balanced.npy`
7. Train/test split	NB03 Cell 2	(412416, 224)	`train_test_split(test_size=0.2, random_state=42, stratify=y)` .	Train (329932, 224), Test (82484, 224)	in-memory (persisted later as `y_train.npy` / `y_test.npy`)
8a. Autoencoder	NB03 Cells 3–4	(329932, 224)	`FraudAutoencoder` : enc `224→256→128→64` (BN+ReLU+Dropout 0.2 ×2), dec mirrors back to 224. Adam, lr=1e-3, 30 epochs, batch 256, MSE loss.	Latent (329932, 64); final recon loss 0.153243	`models/autoencoder.pt` , `data/ae_loss.png`

8b. ResNet extractor	NB03 Cells 5–7	(329932, 224)	`FraudResNet`: stem `Linear(224,128)` → 4× `ResBlock` (128→128→64→64→64) → head `Linear(64,2)`. Adam lr=1e-3, 20 epochs, batch 256, CrossEntropyLoss (2-class fraud/legit).	Features (329932, 64) via `.features()`; final CE loss 0.0942	`models/resnet_extractor.pt`, `data/resnet_loss.png`
9. Concatenate	NB03 (implicit — see §11)	AE latent (·,64) + ResNet features (·,64)	`np.concatenate([z_ae, z_rn], axis=1)` → `Z_train_concat` / `Z_test_concat` — **this assembly step is not present in the saved notebook cells; the variable is used but never defined (§11)**.	(·, 128)	in-memory only
10. Nystroem	NB03 Cell 8	(329932, 128)	`Nystroem(kernel='rbf', gamma=0.05, n_components=300, random_state=42)`, fit on a random 5,000-row sample; batched `transform()` (10,000-row chunks) to avoid an O(n²) kernel matrix.	(329932, 300) train, (82484, 300) test	`models/nystroem.pkl`
11. IncrementalPCA	NB03 Cell 8	(329932, 300)	`IncrementalPCA(n_components=50, batch_size=10000)`.	(329932, 50) = `Z_train_earn` ("EARN+"), (82484, 50) test	`models/incremental_pca.pkl`, `data/Z_train_earn.npy`, `data/Z_test_earn.npy`
12. Cluster sanity check	NB03 Cell 9	5,000-row sample of (·,50)	`KMeans(n_clusters=2, random_state=42, n_init=10)` → silhouette + adjusted-Rand-index vs. true labels.	Silhouette 0.7784, ARI −0.0001	`data/kpca_clusters.png`
13. AttentionRXTJ training	NB04 Cells 2–7	(329932, 50)	`ResNeXtExtractor(50)→64` feeding `SelfAttentionGRU(64,64,seq_len=8)`. `AdamW` lr=5e-4, weight_decay=1e-4, `CosineAnnealingLR(T_max=40)`, 40 epochs, batch 256, `BCEWithLogitsLoss(pos_weight=1.0)`, grad-norm clip 1.0. Best-checkpoint kept by validation AUC.	Test probs (82484,); Best Val AUC 0.9682	`models/attention_rxtj.pt`, `data/training_curves.png`, `data/roc_curve.png`, `data/confusion_matrix.png`
14. Isolation Forest	NB04 Cells 9–10	Fraud-only rows of (·,50)	`IsolationForest(n_estimators=200, contamination=0.05, max_samples='auto', random_state=42, n_jobs=-1)`, fit only on the 164,966 fraud-labeled training rows ("learns the fraud manifold"). Score min-max normalized and inverted (higher = more anomalous).	IFM scores (82484,) in [0,1]	`models/isolation_forest.pkl`, `data/model_probs.npy`, `data/ifm_scores.npy`
15. Jaya weight optimization	NB05 Cells 2–4	`model_probs.npy`, `ifm_scores.npy`, `y_test.npy`	Jaya metaheuristic, population 30, 100 iterations, over `(W_MODEL, W_IFM)∈[0,1]²`, minimizing `0.5·(1-AUC)+0.3·fiscal_cost+0.2·(1-MCC)` where `fiscal_cost=(2·FP+1·FN)/n`. Update rule: `X_new = X + r1·(X_best−|X|) − r2·(X_worst−|X|)`, clipped to bounds, greedy replace-if-better.	Raw weights [1.0, 0.8716] → normalized W_MODEL=0.5343, W_IFM=0.4657	`results/jaya_convergence.png`
16. Threshold sweep	NB05 Cell 6	Fused scores, thresholds 0.10–0.89 step 0.01	Grid search minimizing `fiscal_cost` at each threshold.	Optimal threshold = 0.50	`results/threshold_optimisation.png`

17. Final evaluation	NB05 Cell 7	(82484,) fused scores at threshold 0.50	ROC-AUC, MCC, confusion matrix.	AUC 0.9624, MCC 0.8426, FP 2,748, FN 3,753, TP 37,489, TN 38,494	`results/roc_comparison.png`, `results/confusion_matrix_final.png`
18. Save deployment config	NB05 Cell 9	All of the above	Assembles the final dict (weights, threshold, metrics, benchmarked latency/throughput — see §11.1 caveat).	JSON	`results/deployment_config.json`, `models/jaya_optimal_weights.pkl`

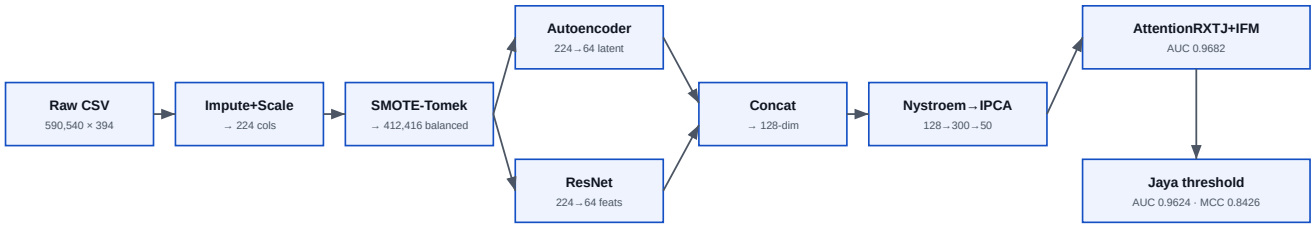


Figure 5.1 — Phase 1 training pipeline: raw CSV to Jaya-optimized fraud score (NB01–05).

NOTEBOOK 01 IS A STUB

`notebooks/01_preprocessing.ipynb` contains exactly one code cell (unexecuted — no `execution_count`, no outputs) illustrating mean imputation + standardization on *numeric-only* columns via `df.select_dtypes(include='number')`, and it performs no `joblib.dump`. The `imputer.pkl` / `scaler.pkl` artifacts actually shipped in `models/` are produced by **NB02 Cell 10**, fit on the label-encoded 224-column matrix (which includes the categorical columns NB01's stub explicitly excludes). NB01 should be read as an early illustrative sketch, not the source of the deployed preprocessing artifacts.

5.2 Phase 2 Training Pipeline (notebooks 06–08)

5.2.1 NB06 — Behavioral profiling & snapshot generation

Inputs	`data/IEEE CIS/train_transaction.csv` (columns: `TransactionID, isFraud, TransactionDT, TransactionAmt, ProductCD, card1, addr1`), `train_identity.csv` (`TransactionID, DeviceInfo`, left-merged; if absent, `DeviceInfo` is set to null).
Account grouping	Rows sorted by `(card1, TransactionDT)`; `card1` is treated as the account identifier throughout (cast to `str(int(card1))`, or `"unknown"` if null).
Profile creation	For each row, in time order, `ProfileStore.update_profile(acct, txn)` is called (same method documented in §3.6) — this is the *identical code path* the live API uses, so the historical profile DB and the live-updated profiles share exactly one update rule.
EMA updates	α=0.1 exponential moving average on amount mean/std (see §3.6/§14 formula); hour-of-day histogram renormalized EMA; merchant/product-code counters; known-device list (capped at 50, most-recent-first).
Velocity calculations	Computed **separately** from `ProfileStore`, vectorized over the full sorted transaction table: for each row, `velocity_1h` / `velocity_24h` = count of that

account's prior transactions (look-back window capped at 200 rows for speed) within the trailing 3,600s / 86,400s window. These values are written only into `tx_snapshot.parquet`, **not** into the `profiles` table (whose `velocity_1h` / `velocity_24h` columns stay at their INSERT-time default of 0 for every account — see §11).	Device novelty
Per `(card1, device_string)` pair, the first occurrence (by `TransactionDT`) is marked `1.0` (novel), later occurrences `0.0` (seen before), and empty device strings `0.5` (neutral/unknown). Header comment documents this as a deliberate fix replacing a slower/broken `expanding().apply()`-on-strings approach.	Geographic behavior
Not computed in NB06; `geo_cluster` in the profile is simply the most recent non-empty `addr1` string (see §3.6, §14 F3 formula, computed downstream in NB07/`app.py`).	Merchant behavior
Per-account `merchant_counts` dict keyed by `ProductCD`, incremented on each transaction (via `ProfileStore`); a separate, dataset-wide per-`ProductCD` fraud rate is computed later in NB07 (`merchant_fraud_rate` feature, not part of the profile row).	Profile persistence
SQLite `profiles` table via `ProfileStore` — schema in §9.1.	Snapshot generation
15-column `tx_snapshot.parquet` (schema in §9.4) written once, before the device-novelty computation, "so partial runs don't lose data" (header comment).	Smart resume / checkpoint logic
See §10.6 for the full mechanism — gated by `ProfileStore.coverage_stats()['total_accounts'] >= 0.9 × unique_account_count`.	Outputs

`data/behavioral_profiles.db`, `data/tx_snapshot.parquet` (590,540 rows), `data/profile_stats.json`.

5.2.2 NB07 — Phase 1 scores → contextual features → labels

Inputs	`data/tx_snapshot.parquet`, `models/imputer.pkl`, `scaler.pkl`, `autoencoder.pt`.
Phase 1 scores (feature F8)	**Not computed in this notebook.** Column 7 (`p1_risk_score`) is written as a constant `0.5` placeholder for all 590,540 rows (mean=0.500, std=0.000, correlation with label = 0.0000), with the inline comment "placeholder 0.5 (NB08 will patch)". See §11.1 for what happens when NB08 attempts the patch.
Contextual feature generation	All 13 features computed vectorized/grouped over the snapshot — see the per-feature table below.
Account-compromise labels	`account_compromised = isFraud==1 OR (24h before that account's first confirmed fraud AND isFraud==0)`. Raises positives from 20,663 (3.50%, transaction-level `isFraud`) to 23,400 (3.96%) by adding 2,737 "probe window" transactions.
Outputs	`data/contextual_features.npy` (590540, 13), `data/fusion_labels.npy`, `data/fusion_account_ids.npy`, `data/fusion_feature_names.json`, `data/drift_norm_params.json`, `data/merchant_fraud_rates.json`.

THE 13 CONTEXTUAL FEATURES (AS COMPUTED IN NB07, FORMULAS IDENTICAL TO `APP.PY COMPUTE_CONTEXTUAL_FEATURES`)

#	Name	Formula	Input	Output range	Label corr. (NB07)
0	`amount_z_score`	`clip((amount − amt_cumsum) / max(amt_cumstd, 0.1), −5, 5)`	Expanding per-account mean/std of amount, shifted by 1 row	[−5, 5]	+0.0248
1	`merchant_novelty`	`1 − seen_count(ProductCD)/j` for the *j*-th transaction of the account (1.0 on first occurrence of a category)	Per-account running ProductCD counts	[0, 1]	+0.0148
2	`geo_displacement`	`clip(|addr1 − expanding_median(addr1, shift 1)| / 500, 0, 1)`	Per-account expanding median of `addr1`	[0, 1]	−0.0045
3	`hour_deviation`	`1 − seen_count(hour)/j` (analogous to F1, over the 24 hour bins)	Per-account running hour counts	[0, 1]	+0.0036
4	`device_novelty`	1.0 first time a `(card1, device)` pair is seen, 0.0 thereafter, 0.5 if no device string	First-seen timestamp per device key	{0, 0.5, 1}	−0.0480
5	`velocity_ratio`	`clip(velocity_1h / (velocity_24h/24 + 1e-6), 0, 10)`	Snapshot velocity columns (NB06)	[0, 10]	+0.0529
6	`behavioral_drift`	`clip((AE_recon_MSE − p5) / (p95 − p5), 0, 1)` — autoencoder reconstruction error, normalized against the dataset's 5th/95th percentile	4-column raw vector (amount, hour, ProductCD-code, addr1) → impute → scale → AE forward pass	[0, 1]	+0.1405 (strongest positive of F1–F7)

7	`p1_risk_score`	Phase-1 ensemble score	—	constant 0.5 in NB07 (patched by NB08 — see §11.1)	0.0000
8	`amount_percentile`	`expanding_rank(pct=True, shift 1)` of amount within the account, filled 0.5 for no history	Per-account expanding rank	[0, 1]	+0.0012
9	`time_gap_norm`	`clip(log1p(hours_since_last_txn) / p99(log-gap), 0, 1)`	Consecutive `TransactionDT` deltas per account	[0, 1]	−0.0420
10	`merchant_fraud_rate`	Per-`ProductCD` mean of `isFraud` over the whole dataset (static lookup)	`data/merchant_fraud_rates.json`	[0.020, 0.117]	+0.1579 (strongest of all 13)
11	`cross_device_accounts`	`clip(distinct_account_count(device) / p99(that count), 0, 1)` — how many different accounts share this device	Per-device account-count table	[0, 1]	+0.0516
12	`account_age_score`	`1 / (1 + txn_seq)` — inverse of the account's 0-based transaction sequence number (1st txn=1.0, 10th≈0.09, 50th≈0.02)	Per-account cumulative count	(0, 1]	−0.0127

PANDAS FUTUREWARNING

Two of the per-account grouped computations (`merchant_novelty` Cell 5 and `hour_deviation` Cell 5, and again `time_gap_norm` Cell 8) call `tx.groupby("card1", group_keys=False).apply(fn)` where `fn` reads the grouping column (`card1`) from inside the group. Pandas prints, verbatim: `DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass include_groups=False...` — 3 occurrences confirmed in the saved cell output. See §11.2.

5.2.3 NB08 — FusionNet v3 training

Inputs	`data/contextual_features.npy`, `fusion_labels.npy`, `fusion_account_ids.npy`, `fusion_feature_names.json`, `tx_snapshot.parquet` (for `TransactionDT`), `train_transaction.csv` (all 224 named columns, for Stage 1 P1 regeneration), all 8 Phase-1 model artifacts.
Stage 1 — P1 score regeneration	A guard checks whether a cached `data/model_probs_full.npy` exists with matching row count **and** `std > 0.05`; if not, it deletes the stale file and recomputes P1 scores end-to-end over the *full* 224-named-column CSV (not the 4-column proxy used by NB07) through the entire Phase-1 pipeline (impute→scale→AE+ResNet→concat→Nystroem→IPCA→AttentionRXTJ+IFM). The freshly regenerated scores are then written back into `contextual_features.npy` column 7. **The regenerated scores measured `std=0.0114` — below the same 0.05 threshold that triggered regeneration — and are used unmodified anyway** (root-caused in §11.1).
Temporal split	70th/85th percentile of `TransactionDT` define train/val/test boundaries (not random, not stratified): Train 413,378 rows (4.16% positive), Val 88,581 rows (3.49% positive), Test 88,581 rows (3.51% positive).
Feature preparation	`StandardScaler` fit on the train split only, applied to val/test (no leakage) → `models/fusion_scaler.pkl`.

Model training	`FusionNet(input_dim=13)`; loss `FocalLoss(alpha=0.75, gamma=2.0)`; optimizer `Adam(lr=3e-4, weight_decay=1e-4)`; scheduler `CosineAnnealingLR(T_max=200, eta_min=1e-5)`; batch size 4,096; up to 200 epochs; early stopping patience 25 epochs (never triggered — trained the full 200 epochs, "Best val AUC" reached only at epoch ~140–200 as the plateau slowly improves 0.7343→0.7354); gradient-norm clip 1.0. Wall-clock training time printed: 7,063s (~1 h 58 min) for Stage 2 alone.
Threshold calculation	A second, smaller Jaya-style search (population 40, 150 iterations) over the validation predictions, minimizing `2·FPR + FNR`, yields `fusion_threshold=0.3574`; `high_threshold = min(opt+0.15, 0.90) = 0.5074`; `elevated_threshold = max(opt−0.10, 0.25) = 0.2574`.
Metrics (held-out test set)	AUC 0.7406, PR-AUC 0.1028, MCC 0.1862, Precision 0.1193, Recall 0.4835, F1 0.1914, FPR 0.1300, confusion matrix TP=1505 FP=11112 TN=74356 FN=1608.
Saved artifacts	`models/fusion_net.pt`, `models/fusion_scaler.pkl`, `results/fusion_config.json`, `results/fusion_roc.png`, `results/fusion_training_curves.png`.

CONFIRMED — SELF-DOCUMENTED TARGET MISS

NB08's own printed evaluation panel states four explicit targets and shows the achieved value fails to meet **every one of them**: AUC 0.7406 vs. "target ≥ 0.85"; MCC 0.1862 vs. "target ≥ 0.35"; Precision 0.1193 vs. "target ≥ 0.15" (missed by 0.0007); FPR 0.1300 vs. "target ≤ 0.10". The notebook's own comment block at the top predicted "AUC: 0.83–0.90 (was 0.7537), MCC: 0.35–0.55 (was 0.1708), FPR: <10% (was 15.49%)" — the actual run improved FPR modestly (15.49%→13.00%) but AUC/MCC did not reach the predicted range. Full root-cause analysis in §11.1.

5.3 Inference Pipeline — `POST /score/form-full`

Verified line-by-line against `app.py:786-921`. This is the only endpoint that exercises the complete Phase 1 + Phase 2 pipeline starting from plain human-readable form fields (no pre-computed feature vector).

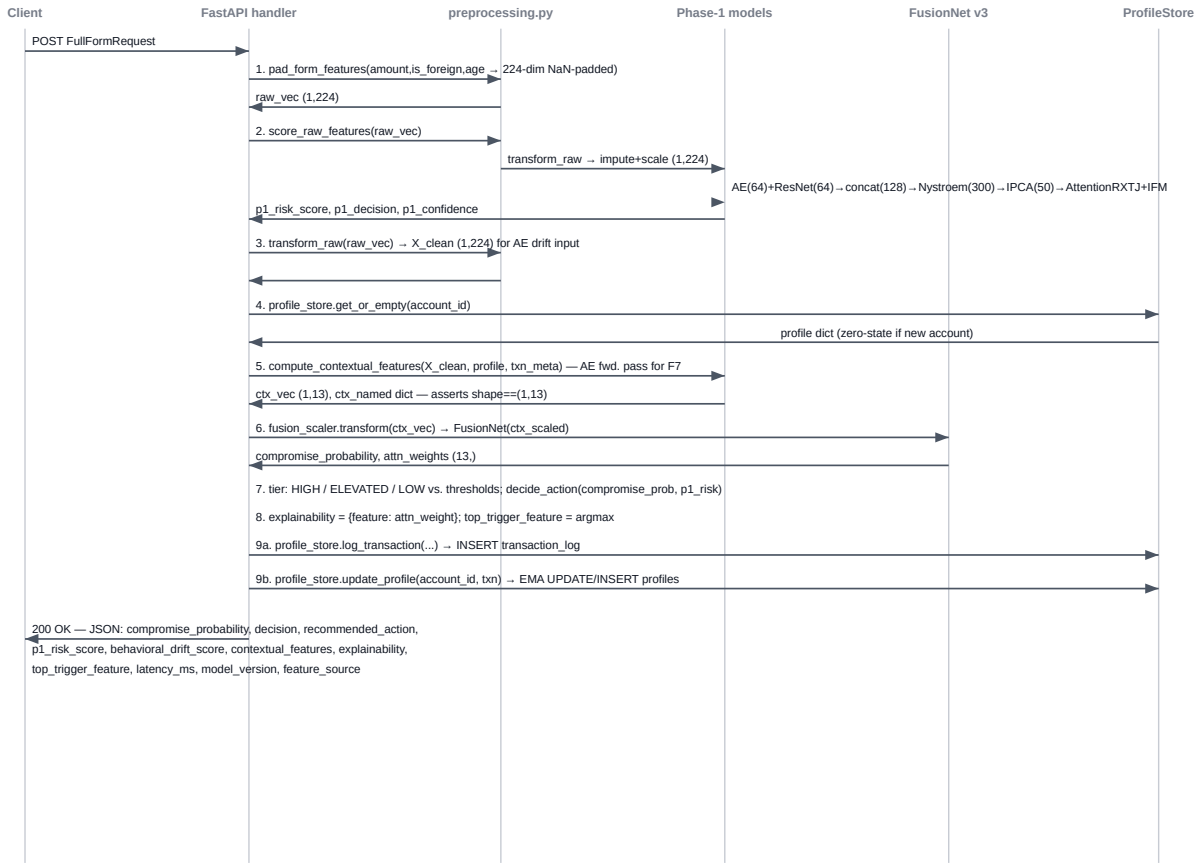


Figure 5.2 — Sequence diagram for `POST /score/form-full`, the single endpoint that runs the entire Phase 1 + Phase 2 pipeline from raw form fields (app.py:786-921).

5.3.1 Step-by-step, with exact source references

1. **Request parsing / validation** — Pydantic validates `FullFormRequest` (§3.4): `hour` constrained to [0,23], `customer_age` to [0,120]; all other fields unconstrained strings/floats. Invalid types → automatic FastAPI 422 response (framework behavior, no custom handler).
2. **Feature extraction / padding** (`app.py:812-817`) — builds a 3-element list `[amount, 1.0 if is_foreign else 0.0, customer_age or 30]` and calls `pad_form_features(partial, RAW_DIM=224)`, which places these 3 values at raw-feature columns 0–2 and leaves the remaining 221 columns as `NaN`.
3. **Preprocessing** (`app.py:820-821`) — `score_raw_features(raw_vec)` calls `transform_raw` (imputer fills the 221 NaN columns with training-set means, then `StandardScaler.transform`).
4. **Phase-1 scoring** (same call) — `extract_earn_features` (AE + ResNet → concat → Nystroem → IPCA) then `score_earn_features` (AttentionRXTJ probability + IFM anomaly score, Jaya-weighted fusion, threshold at 0.50) → `p1_risk_score`, `p1_decision`, `p1_confidence`.
5. **AE reconstruction / drift input** (`app.py:826-828`) — a *second* call to `transform_raw` on the same `raw_vec` produces `X_clean`, the 224-dim input the autoencoder needs for the behavioral-drift feature (this recomputes the identical imputer/scaler transform already done inside step 3/4 — a redundant call, not an error, since both calls are deterministic and cheap).
6. **Profile retrieval** (`app.py:831`) — `profile_store.get_or_empty(account_id)`; SQLite read.
7. **Contextual feature calculation** (`app.py:833-850`) — `compute_contextual_features(X_clean, profile, txn_meta)` where `txn_meta` bundles the form fields plus the just-computed `p1_risk_score` and a timestamp

(request-supplied or `time.time()`); asserts the output shape is exactly `(1,13)`, raising an HTTP 500 with an explicit message if not.

8. **Phase-2 scoring** (app.py:852-859) — `fusion_scaler.transform(ctx_vec)` then `fusion_model(ctx_tensor)` → `(compromise_prob, attn_weights)`.
9. **Threshold application / action selection** (app.py:862-869) — 3-tier decision (HIGH ≥ 0.5074, ELEVATED ≥ 0.2574, else LOW) plus `decide_action(compromise_prob, p1_risk)` (§14 table).
10. **Explainability** (app.py:871-876) — per-feature attention weight dict; `top_trigger_feature = argmax`.
11. **Database logging** (app.py:878-888) — `profile_store.log_transaction(...)`, one `INSERT` into `transaction_log`.
12. **Profile update** (app.py:889-896) — `profile_store.update_profile(account_id, {...})`, EMA blend + `UPDATE` / `INSERT` into `profiles`.
13. **Response generation** (app.py:898-916) — assembles the JSON body (field list in §8.4); wraps the whole handler body in `try/except HTTPException: raise / except Exception: HTTPException(500, str(e))`.

6. Model Architectures

Every artifact below was deserialized directly (`torch.load(..., map_location='cpu')` for `.pt` files, `joblib.load(...)` for `.pkl` files) and its tensor shapes / fitted attributes read back programmatically. Values are exact, not estimated.

6.1 models/attention_rxtj.pt — `AttentionRXTJ`

Python class	`AttentionRXTJ(input_dim=50, seq_len=8)` — `app.py:88-94`
Serialization	Raw PyTorch `state_dict` (`collections.OrderedDict`), 244 tensor keys
Input shape	(n, 50) — the EARN+ feature vector
Output shape	`(logit: (n,), attn_weights: (n, 8))`
Total parameters (from state_dict tensors)	107,362

Sub-module	Layer-by-layer (state_dict key pattern → shape)	Notes
`resnext.net.0` *ResNeXtBlock(50→128)*	4× path `{0.weight:[32,50], 0.bias:[32], 1.weight/bias/running_mean/running_var:[32]}` (BatchNorm) → `{3.weight:[32,32], 3.bias:[32], 4.*:[32]}` (BatchNorm); `shortcut.weight:[128,50], shortcut.bias:[128]`	4 parallel cardinality paths, group_dim=32, concatenated → 128; shortcut is a full Linear projection since in_dim(50)≠out_dim(128).
`resnext.net.1` *ResNeXtBlock(128→128)*	4× path `{0.weight:[32,128], ...3.weight:[32,32]...}`; `shortcut` = `nn.Identity()` (no weight — in_dim==out_dim)	
`resnext.net.2` *ResNeXtBlock(128→64)*	4× path `{0.weight:[16,128] ...3.weight:[16,16]}`; `shortcut.weight:[64,128], shortcut.bias:[64]`	group_dim=16.
`resnext.net.3` *ResNeXtBlock(64→64)*	4× path `{0.weight:[16,64] ...3.weight:[16,16]}`; `shortcut` = Identity	ResNeXt output: 64-dim.
`attn_gru.gru` *nn.GRU(8,64,num_layers=2,dropout=0.3)*	`weight_ih_l0:[192,8], weight_hh_l0:[192,64], bias_ih_l0/bias_hh_l0:[192]`; `weight_ih_l1:[192,64], weight_hh_l1:[192,64], bias_ih_l1/bias_hh_l1:[192]`	192 = 3×64 (GRU has 3 gates); layer-1 input is layer-0's 64-dim hidden output. `step_dim = 64/8 = 8`.
`attn_gru.attention` *Linear(64,32)→Tanh→Linear(32,1)*	`0.weight:[32,64], 0.bias:[32]; 2.weight:[1,32], 2.bias:[1]`	Produces one scalar score per GRU timestep, softmax-normalized over the 8 timesteps.
`attn_gru.classifier` *Linear(64,32)→ReLU→Dropout(0.3)→Linear(32,1)→Identity*	`0.weight:[32,64], 0.bias:[32]; 3.weight:[1,32], 3.bias:[1]`	Index-4 `Identity` has no parameters (dropped sigmoid; caller applies it).

6.2 models/autoencoder.pt — `AutoencoderNet`

Python class	`AutoencoderNet(input_dim=224, latent_dim=64)` — app.py:96-111
Input / output shape	(n, 224) → `(reconstruction: (n,224), latent: (n,64))`
Total parameters	200,740

state_dict key	Source-code layer	Shape
`encoder.0.weight / .bias`	`encoder[0] = Linear(224,256)`	[256,224] / [256]
`encoder.1.*` (weight,bias,running_mean,running_var)	`encoder[1] = BatchNorm1d(256)`	[256] each
`encoder.4.weight / .bias`	`encoder[4] = Linear(256,128)`	[128,256] / [128]
`encoder.5.*`	`encoder[5] = BatchNorm1d(128)`	[128] each
`encoder.8.weight / .bias`	`encoder[8] = Linear(128,64)`	[64,128] / [64]
`decoder.0.weight / .bias`	`decoder[0] = Linear(64,128)`	[128,64] / [128]
`decoder.1.*`	`decoder[1] = BatchNorm1d(128)`	[128] each
`decoder.3.weight / .bias`	`decoder[3] = Linear(128,256)`	[256,128] / [256]
`decoder.4.*`	`decoder[4] = BatchNorm1d(256)`	[256] each
`decoder.6.weight / .bias`	`decoder[6] = Linear(256,224)`	[224,256] / [224]

Activations: ReLU after every Linear except the final decoder output and the encoder's last Linear (which is followed by ReLU per source, i.e. latent is non-negative). **Dropout:** 0.2 after the two hidden encoder blocks only (none in the decoder). **Normalization:** BatchNorm1d on every hidden layer. **Training objective:** MSE reconstruction loss, Adam lr=1e-3, 30 epochs, batch 256 (NB03). This same artifact is reused at inference for two distinct purposes: (1) the 64-dim latent as half of the EARN+ 128-dim concat input, and (2) the reconstruction MSE as the `behavioral_drift` Phase-2 feature.

6.3 models/resnet_extractor.pt — `FraudResNet`

Python class	`FraudResNet(input_dim=224, out_dim=64)` — app.py:127-142
Input / output shape	(n, 224) → `.features()`: (n, 64); `.forward()`: (n, 2) (unused at inference)
Total parameters	101,834

Sub-module	Shapes	Notes
`stem`	`weight:[128,224], bias:[128]`	`Linear(224,128)`, followed by a bare `torch.relu()` call (not a module — no state_dict key) before entering `blocks`.
`blocks.0` *ResBlock(128→128)*	`block.0:[128,128]/[128], block.1(BN):[128]×4, block.3:[128,128]/[128], block.4(BN):[128]×4`	`shortcut` = Identity (dims equal).
`blocks.1` *ResBlock(128→64)*	`block.0:[64,128]/[64] ... block.3:[64,64]/[64] ...`; `shortcut.weight:[64,128], shortcut.bias:[64]`	Only block with a projecting shortcut.

`blocks.2` *ResBlock(64→64)*	`block.0: [64,64]/[64] ...`	`shortcut` = Identity.
`blocks.3` *ResBlock(64→64)*	`block.0: [64,64]/[64] ...`	`shortcut` = Identity.
`head`	`weight:[2,64], bias:[2]`	`Linear(64,2)` — original 2-class training head; `app.py` never calls `.forward()`, only `.features()`, so this layer is loaded but dead weight at inference.

Training objective: `CrossEntropyLoss` (2-class fraud/legit) on the `.forward()` head, Adam lr=1e-3, 20 epochs, batch 256 (NB03).

6.4 models/fusion_net.pt — `FusionNet` (v3)

Python class	`FusionNet(input_dim=13)` — `app.py:153-180`
Input / output shape	(n, 13) → `(compromise_prob: (n,), attn_weights: (13,))`
Total parameters	12,174

state_dict key	Source-code layer	Shape
`feature_attn`	`nn.Parameter(torch.ones(13))`	[13] — learnable, softmax-normalized per-feature gate
`net.0.weight / .bias`	`net[0] = Linear(13,128)`	[128,13] / [128]
`net.3.weight / .bias`	`net[3] = Linear(128,64)`	[64,128] / [64]
`net.6.weight / .bias`	`net[6] = Linear(64,32)`	[32,64] / [32]
`net.8.weight / .bias`	`net[8] = Linear(32,1)`	[1,32] / [1]

Activations: ReLU after `net[0]` and `net[3]` and `net[6]`; final sigmoid applied outside `net` in `forward()`. **Dropout:** 0.3 after `net[0]`'s ReLU, 0.3 after `net[3]`'s ReLU, none after `net[6]`. **Attention:** input `x` is element-wise multiplied by `softmax(feature_attn)` *before* entering `net` — this is feature-level (not layer-level) attention, and its 13 weights are exactly what is returned to the API caller as the `explainability` dict. **Training:** Focal Loss (α=0.75, γ=2.0), Adam lr=3e-4, weight_decay 1e-4, CosineAnnealingLR (T_max=200, eta_min=1e-5), batch 4096, up to 200 epochs (NB08). **Thresholds:** fusion 0.3574, elevated 0.2574, high 0.5074 (`results/fusion_config.json`).

6.5 scikit-learn / joblib artifacts

File	Estimator	Fitted parameters	Feature counts	How loaded / consumed

`imputer.pkl`	`SimpleImputer`	`strategy='mean', missing_values=nan, fill_value=None, add_indicator=False, keep_empty_features=False`	`n_features_in_=224`; `statistics_` is a length-224 float64 array of per-column training means; `feature_names_in_` holds the 224 original CSV column names (first 5: `TransactionDT, TransactionAmt, ProductCD, card1, card2`)	`preprocessing.load_preprocessors` → `app.py`; reloaded independently in NB07 & NB08
`scaler.pkl`	`StandardScaler`	`with_mean=True, with_std=True`; `mean_` / `scale_` / `var_` length-224 float64 arrays; `n_samples_seen_=590540`	224 features	same as above
`nystroem.pkl`	`Nystroem`	`kernel='rbf', gamma=0.05, n_components=300, random_state=42`; `components_` float32 array (300,128) — the sampled basis rows	`n_features_in_=128` in → 300 out	`app.py extract_earn_features`; NB05 benchmark; NB08 Stage 1
`incremental_pca.pkl`	`IncrementalPCA`	`n_components=50, batch_size=10000, whiten=False`; `components_` (50,300); `mean_` / `var_` (300,); `explained_variance_ratio_` top value 0.1876; `n_samples_seen_=329932`	`n_features_in_=300` → `n_components_=50`	same as Nystroem consumers
`isolation_forest.pkl`	`IsolationForest`	`n_estimators=200, contamination=0.05, max_samples='auto'(→256), bootstrap=False, random_state=42, n_jobs=-1`; `offset_=-0.4634`	`n_features_in_=50`	`app.py score_earn_features` (via `.score_samples`); NB08 Stage 1 (via `.decision_function` — a different sklearn method, see §11)
`kernel_pca.pkl`	`KernelPCA`	`kernel='rbf', gamma=0.05, n_components=50, degree=3, coef0=1, random_state=42, eigen_solver='auto', fit_inverse_transform=False`	`n_features_in_=128`	*Orphaned — dumped by NB03 but never loaded anywhere (§2.3, §11).*
`fusion_scaler.pkl`	`StandardScaler`	`with_mean=True, with_std=True`; `mean_` / `scale_` / `var_` length-13; `n_samples_seen_=413378` (train-split size only — confirms it was fit post-split, not leaked)	13 features	`app.py` — scales the 13-dim contextual vector immediately before FusionNet
`jaya_optimal_weights.pkl`	raw `numpy.ndarray`	`[1.0, 0.87162169]` — the **unnormalized** Jaya solution before the W_MODEL/W_IFM normalization performed inline in NB05	—	*Orphaned — `app.py` reads the pre-normalized weights from `deployment_config.json` instead.*

6.6 Serialization formats summary

Format	Files	Loader
Raw PyTorch `state_dict` (pickle, via `torch.save`)	`attention_rxtj.pt`, `autoencoder.pt`, `fusion_net.pt`, `resnet_extractor.pt`	`torch.load(path, map_location='cpu')` then `module.load_state_dict(...)`
joblib pickle	all 8 `.pkl` files	`joblib.load(path)`

7. Configuration & Artifacts

Every JSON configuration file in the repository, with every key documented. No YAML, TOML, INI, or `.env` files exist anywhere in the repository (confirmed by full-tree search); all runtime configuration is either a JSON file under `data/` / `results/` or a Python module-level constant inside `app.py` / `profile_store.py` (catalogued in §3).

7.1 results/deployment_config.json


```
{
  "W_MODEL": 0.5342960077804467,
  "W_IFM": 0.4657039916852572,
  "THRESHOLD": 0.4999999999999998,
  "final_auc": 0.9624399631394758,
  "final_mcc": 0.8426196473448768,
  "fiscal_cost": 0.11213083749575675,
  "false_positives": 2748,
  "false_negatives": 3753,
  "true_positives": 37489,
  "true_negatives": 38494,
  "throughput_tps": 51351.761002670246,
  "latency_ms": 0.01947352886199951,
  "cost_fp_weight": 2.0,
  "cost_fn_weight": 1.0
}
```


Key	Type	Meaning	Producer	Consumer	Static/generated
`W_MODEL`	float	Jaya-optimized, normalized weight for the AttentionRXTJ probability in the Phase-1 fusion sum.	NB05 Cell 9	`app.py` L234, used in `score_earn_features`	Generated (Jaya search)
`W_IFM`	float	Jaya-optimized, normalized weight for the IsolationForest anomaly score.	NB05 Cell 9	`app.py` L235	Generated
`THRESHOLD`	float	Decision threshold on the fused Phase-1 score (found to equal exactly 0.50 by the sweep).	NB05 Cell 6	`app.py` L236	Generated
`final_auc`	float	Phase-1 test ROC-AUC at the final weights/threshold.	NB05 Cell 7	`app.py` `/`, `/model/info`	Generated
`final_mcc`	float	Phase-1 test Matthews correlation coefficient.	NB05 Cell 7	`app.py` `/model/info`	Generated
`fiscal_cost`	float	Normalized weighted misclassification cost, `(2·FP+1·FN)/n`, at the final threshold.	NB05 Cell 7	Not read by `app.py`; documentation only.	Generated

`false_positives` / `false_negatives` / `true_positives` / `true_negatives`	int	Confusion-matrix cell counts on the 82,484-row Phase-1 test split.	NB05 Cell 7	`app.py` `/model/info`	Generated
`throughput_tps`, `latency_ms`	float	Benchmarked scoring speed. **Caveat:** measured over `Nystroem.transform → IncrementalPCA.transform → IsolationForest.score_samples` on 1,000 rows of `np.random.randn(1000,128)` synthetic noise (NB05 Cell 9) — *excludes* the AttentionRXTJ forward pass, the Autoencoder, the ResNet extractor, and the imputer/scaler, and does not run against real preprocessed transaction data. See §11.1.	NB05 Cell 9	`app.py` `/`, `/model/info`	Generated (benchmark)
`cost_fp_weight`, `cost_fn_weight`	float	Fixed cost multipliers used by the Jaya objective (`COST_FP=2.0, COST_FN=1.0`) — false positives penalized twice as heavily as false negatives.	NB05 Cell 1 (hand-set constant)	Not read by `app.py`; documentation only.	Static (hand-tuned)

7.2 results/fusion_config.json


```
{
  "input_dim": 13, "fusion_threshold": 0.357353, "high_threshold": 0.507353,
  "elevated_threshold": 0.257353, "fusion_auc": 0.740583, "fusion_pr_auc": 0.102779,
  "fusion_mcc": 0.186221, "fusion_precision": 0.119284, "fusion_recall": 0.483456,
  "fusion_f1": 0.191354, "fusion_fpr": 0.130014, "true_positives": 1505,
  "false_positives": 11112, "true_negatives": 74356, "false_negatives": 1608,
  "feature_names": [ ...13 names... ],
  "optimal_attn_weights": [0.07808, 0.078173, 0.066759, 0.068697, 0.066629,
    0.081783, 0.066497, 0.070814, 0.067208, 0.078804, 0.13657, 0.067021, 0.072963],
  "model_version": "fusionnet_v3_focal_temporal", "training_epochs": 200,
  "best_val_auc": 0.735392, "loss_fn": "FocalLoss(alpha=0.75, gamma=2.0)",
  "split_method": "temporal (not random)",
  "label_strategy": "account_compromise_window_24h"
}
```


Key	Type	Meaning	Consumer	Static/generated
`input_dim`	int (13)	FusionNet input width — used by `app.py` to construct `FusionNet(f_input_dim)` rather than hard-coding 13 (`app.py:218-219`).	`app.py` `load_models`	Generated (= feature count)

`fusion_threshold`, `high_threshold`, `elevated_threshold`	float	Jaya-searched decision boundaries. `app.py` uses `high_threshold` / `elevated_threshold` for the live 3-tier decision (`app.py:862-867`); `fusion_threshold` is loaded into `FUSION_THRESHOLD` (L238) and surfaced only in the `/model/info` response body (`app.py:644`) — it does not itself gate any scoring decision at runtime.	`app.py load_models`	Generated (Jaya search)
`fusion_auc`, `fusion_pr_auc`, `fusion_mcc`, `fusion_precision`, `fusion_recall`, `fusion_f1`, `fusion_fpr`	float	Phase-2 held-out test metrics (§1.12 table; §11.1 root-cause).	`app.py /model/info`	Generated
`true_positives` / `false_positives` / `true_negatives` / `false_negatives`	int	Phase-2 test confusion matrix (n=88,581).	Documentation only.	Generated
`feature_names`	list[str] (13)	Canonical ordered feature-name list — **authoritative** source for `FEATURE_NAMES` at runtime.	`app.py load_models` (`FEATURE_NAMES`), every explainability response	Generated (= NB07's feature order)
`optimal_attn_weights`	list[float] (13)	The *softmax* attention weights learned by `FusionNet.feature_attn` at the end of training, saved for inspection. **Not loaded by `app.py`** — at inference, `app.py` recomputes `softmax(fusion_model.feature_attn)` live from the loaded model weights (`app.py:857-859`), which is mathematically identical to this saved snapshot but is an independent code path.	*Not read by `app.py` — documentation/reproducibility snapshot only.*	Generated
`model_version`	str	`"fusionnet_v3_focal_temporal"` — free-text label, not read by `app.py` (which hard-codes its own response-body `model_version` strings, e.g. `"rxtj_plus_v2_form_full"` — see §12.4).	*Documentation only.*	Static label
`training_epochs`	int (200)	Number of epochs actually run before the loop ended (early stopping never triggered).	*Documentation only.*	Generated
`best_val_auc`	float	Best validation-set AUC observed during training (0.735392) — the checkpoint restored before test evaluation.	*Documentation only.*	Generated
`loss_fn`, `split_method`, `label_strategy`	str	Free-text description of the training configuration for reproducibility.	*Documentation only.*	Static labels

7.3 data/drift_norm_params.json


```
{"p5": 0.20556305348873138, "p95": 0.2462635189294815}
```

5th and 95th percentile of the autoencoder's per-row reconstruction MSE, computed once over the full 590,540-row snapshot in NB07 Cell 4. Consumed by `app.py`'s `DRIFT_NORM` constant (`app.py:242`) inside `compute_contextual_features` feature F7: `clip((mse-p5)/(p95-p5), 0, 1)`. Static (fit once at training time; never updated by the running server).

7.4 data/merchant_fraud_rates.json

```
{"C": 0.116873, "H": 0.047662, "R": 0.037826, "S": 0.058996, "W": 0.020399}
```

Per-`ProductCD` historical fraud rate over the full training dataset, computed once in NB07 Cell 9. Consumed by `app.py`'s `MERCHANT_FRAUD_RATES` constant (`app.py:243`) inside feature F11, additionally re-normalized at inference time to [0,1] via a hard-coded `(v-0.020)/(0.117-0.020)` range (`app.py:542-544`) that mirrors — but does not read — the min/max actually present in this file. Static.

7.5 data/profile_stats.json

```
{
  "total_transactions": 590540, "unique_accounts": 13553, "fraud_count": 20663,
  "fraud_rate_pct": 3.499,
  "profile_coverage": {"total_accounts": 13553, "accounts_5plus": 6512,
                       "coverage_pct": 48.0, "avg_txn_count": 43.6},
  "snapshot_path": "f:\\rxtj_phase_2\\data\\tx_snapshot.parquet",
  "generated_at": "2026-06-04T21:27:27"
}
```

Diagnostic snapshot written once by NB06 Cell 9. Not read by `app.py` at runtime — documentation/monitoring only. Note the `snapshot_path` value is a hard-coded Windows absolute path from the machine the notebook was last executed on (`f:\rxtj_phase_2\...`) — a concrete instance of the hard-coded-path issue discussed in §11.3.

7.6 data/fusion_feature_names.json

```
["amount_z_score","merchant_novelty","geo_displacement","hour_deviation","device_novelty",
 "velocity_ratio","behavioral_drift","p1_risk_score","amount_percentile","time_gap_norm",
 "merchant_fraud_rate","cross_device_accounts","account_age_score"]
```

Byte-for-byte identical to `results/fusion_config.json["feature_names"]`. Written by NB07 Cell 14; `app.py` does not read this file — it reads the config copy instead (`app.py:241`), making this file redundant at runtime but useful as a lightweight, config-independent reference.

7.7 Other configuration sources

SEARCH PERFORMED

A full-tree search for `*.yaml, *.yml, *.toml, *.ini, *.cfg, .env*` returned no matches anywhere in the repository. The only non-JSON "configuration" surfaces are: (a) Python module-level constants in `app.py` (`DEVICE, SEQ_LEN, CARDINALITY, HISTORY_DB, BASE`, §3.2) and `profile_store.py` (`_EMA_ALPHA, _DDL`, §3.6); (b) the two environment variables documented in §12.3; (c) `requirements.txt` (§4.1). README's "Configuration" section describes a `.env.example` file with keys like `APP_HOST`, `APP_PORT`, `FUSION_HIGH_THRESHOLD`, `PROFILE_EMA_ALPHA` — **none of these environment variables are read anywhere in the source code**; only `KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_PROFILE_TOPIC`, and (in `profile_consumer.py` only) `KAFKA_CONSUMER_GROUP` are actually consulted via `os.environ.get`.

8. API Specification

Extracted from a complete line-by-line read of `app.py` (1,151 lines). There are 16 routes. Authentication: NONE on every route (see §13). CORS: `allow_origins=["*","null"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"]` (`app.py:319-325`) — fully open. Latency/throughput figures quoted anywhere below are the `deployment_config.json` benchmark values with the caveat in §11.1; no other latency/throughput measurement exists in the repository.

8.1 Endpoint summary

Method	Path	Request	Response (top-level keys)	Auth	Side effects
GET	`/panel`	—	`text/html` (panel.html)	None	None
GET	`/`	—	service, version, status, phase, model_auc, model_mcc, fusion_auc, latency_ms	None	None
GET	`/health`	—	status, models_loaded, fusion_input_dim, feature_count	None	None
GET	`/model/info`	—	phase1{weights,threshold,performance,speed}, phase2{model,input_dim, feature_names,thresholds,performance}	None	None
GET	`/demo-samples`	—	dict of 8 named 50-dim vectors	None	None (file read)
POST	`/score`	`TransactionRequest`	`RiskResponse`	None	Kafka publish (optional)
POST	`/score/form`	`FormScoreRequest`	transaction_id, risk_score, decision, confidence, latency_ms	None	`fraud_storage.db.history` INSERT; Kafka publish
GET	`/history`	query: `limit`	list of {tid,amt,score,status}	None	None (DB read)
POST	`/score/direct`	`DirectScoreRequest`	transaction_id, risk_score, decision, confidence, latency_ms, model_version	None	None
POST	`/score/batch`	`BatchRequest` (≤1000)	results[], total_latency_ms, fraud_count, legit_count	None	None
POST	`/score/form-full`	`FullFormRequest`	see §8.4	None	`behavioral_profiles.db` transaction_log INSERT + profiles UPDATE/INSERT

POST	`/account/compromise-score`	`CompromiseRequest`	see §8.5	None	same DB writes as above
GET	`/account/profile/{account_id}`	path: account_id; query: window	profile dict + profile_age_hours, window_requested, known_device_count	None	None (DB read); 404 if unknown
GET	`/account/history/{account_id}`	path: account_id; query: limit	account_id, record_count, events[]	None	None (DB read)
GET	`/account/alerts`	query: threshold, limit	threshold, alert_count, alerts[], generated_at	None	None (DB read)
POST	`/account/explain`	`ExplainRequest`	account_id, transaction_id, compromise_prob, decision, top_features[], account_baseline, what_triggered	None	None (DB read); 404 if event not found

8.2 Explicitly requested endpoints

GET /health `(app.py:613-620)`

```
@app.get("/health")
def health():
    return {"status": "healthy", "models_loaded": True,
            "fusion_input_dim": fusion_config['input_dim'],
            "feature_count": len(FEATURE_NAMES)}
```

`models_loaded` is a hard-coded literal `True`, not a live check — if the process is running, model load already happened synchronously and unrecoverably at import time (§8.6), so this field cannot ever be `False` while the server answers requests. No request body. Always 200 (no error path). No DB, Kafka, or model inference involved.

GET /model/info `(app.py:622-655)`

Pure config read, no inference. Response schema:

```
{
 "phase1": {"weights": {"W_MODEL": float, "W_IFM": float}, "threshold": float,
   "performance": {"auc": float, "mcc": float, "false_positives": int, "false_negatives": int},
   "speed": {"latency_ms": float, "throughput_tps": float}},
 "phase2": {"model": "FusionNet v3 (13→128→64→32→1)", "input_dim": int,
   "feature_names": [13 strings],
   "thresholds": {"fusion": float, "high": float, "elevated": float},
   "performance": {"auc": float, "pr_auc": float, "mcc": float, "recall": float}}
}
```

GET /account/alerts `(app.py:1067-1077)`

```
@app.get("/account/alerts")
def get_account_alerts(threshold: float = 0.7, limit: int = 50):
    threshold = max(0.1, min(float(threshold), 0.99))
    alerts = profile_store.get_alerts(threshold=threshold, limit=limit)
    return {"threshold": threshold, "alert_count": len(alerts),
            "alerts": alerts, "generated_at": time.time()}
```

`threshold` silently clamped to [0.1, 0.99] — an out-of-range query value never errors, it is coerced. Delegates entirely to `ProfileStore.get_alerts` (§3.6, §9.2), which computes `MAX(compromise_prob)` per account over the trailing 24h and maps it to a recommended action (§9.2 for the exact 3-tier mapping, which differs numerically from `decide_action`'s thresholds — see §11).

POST /score/form-full `(app.py:786-921)`

Fully documented step-by-step in §5.3. Request/response schemas below.

8.3 Request schemas (all Pydantic models, verbatim field lists)

See §3.4 for the exact field list of every Pydantic model (`TransactionRequest` , `DirectScoreRequest` , `BatchRequest` , `FormScoreRequest` , `RiskResponse` , `CompromiseRequest` , `ExplainRequest` , `FullFormRequest`). Validation constraints present: `FullFormRequest.hour` (`ge=0, le=23`), `FullFormRequest.customer_age` (`ge=0, le=120`); no other field carries a Pydantic-level constraint (no length limits on strings, no positivity constraint on `amount` , no enum constraint on `product_cd`). All list-typed fields (`features`) accept any length — dimension checks, where present, happen manually inside the handler body, not via Pydantic (e.g. `/score/direct` checks `Z_earn.shape[1] != ipca.n_components_` and raises 400; `/score` and `/score/batch` perform **no** such check and instead let a shape mismatch propagate into the generic `except Exception` handler as a 500).

8.4 POST /score/form-full — full response schema

```
{
  "account_id": str, "transaction_id": str,
  "compromise_probability": float, "decision": "LOW"|"ELEVATED"|"HIGH",
  "recommended_action": "APPROVE"|"MONITOR_AND_FLAG"|"STEP_UP_AUTHENTICATION"|"FREEZE_AND_NOTIFY",
  "p1_risk_score": float, "p1_decision": "FRAUD"|"LEGIT", "p1_confidence": float,
  "behavioral_drift_score": float,
  "contextual_features": { ...13 named floats... },
  "explainability": { ...13 named floats (softmax attention weights, sum to 1)... },
  "top_trigger_feature": str,
  "latency_ms": float,
  "model_version": "rxtj_plus_v2_form_full",
  "feature_source": "synthesised_from_form_inputs"
}
```

8.5 POST /account/compromise-score — full response schema

```
{
  "account_id": str, "transaction_id": str,
  "compromise_probability": float, "decision": "LOW"|"ELEVATED"|"HIGH",
  "p1_risk_score": float, "behavioral_drift_score": float,
  "contextual_features": {...13...}, "explainability": {...13...},
  "top_trigger_feature": str, "recommended_action": str,
  "latency_ms": float, "model_version": "rxtj_plus_v2_phase2"
}
```

Differs from `/score/form-full` in three ways: (1) accepts a pre-computed `features` vector instead of synthesizing one from form fields (either 50-dim EARN+ or 224-dim raw, auto-detected by length against `ipca.n_components_`); (2) when given the 50-dim path, it cannot recover a true 224-dim input for the autoencoder — it approximates `X_clean` by zero-padding the 50-dim vector into a 224-dim array (`app.py:938-943`), which is *not* a valid inverse of the Nystroem/IPCA transform (IPCA's own `inverse_transform` result, `Z_nys_approx`, is computed at L939 but then discarded — never used); (3) omits `p1_decision` / `p1_confidence` / `feature_source` from the response.

8.6 Error responses & status codes

Code	Trigger	Where
200	Success	All routes, normal path.
404	Explicit `HTTPException(404, ...)`: `/demo-samples` if `data/demo_samples.json` missing; `/account/profile/{id}` if account unknown; `/account/explain` if the (account_id, transaction_id) pair has no logged event.	`app.py:661-664, 1044-1047, 1086-1089`
400	Explicit dimension check on `/score/direct` only (`features` length ≠ `ipca.n_components_`) and on `/score/batch` (`len(transactions) > 1000`).	`app.py:734-738, 758-759`
422	Automatic FastAPI/Pydantic validation error (wrong types, missing required fields, or constraint violations like `hour` outside [0,23]) — framework-generated, no custom handler.	Framework behavior on every POST endpoint.
500	Every other exception inside a route body is caught by a blanket `except Exception as e: raise HTTPException(status_code=500, detail=str(e))`. This includes shape mismatches on `/score` and `/score/batch` (no upfront dimension check there, unlike `/score/direct`), the explicit shape-assertion failures inside `/score/form-full` and `/account/compromise-score` (raised as `AssertionError` in the latter, caught by a dedicated `except AssertionError` clause that also maps to 500).	Every POST handler.

ERROR LEAKAGE

`str(e)` — the raw Python exception message — is returned verbatim in every 500 response's `detail` field. For a shape-mismatch or a sklearn/PyTorch internal error, this can include internal variable names, array shapes, or (for a malformed `KeyError` on a config dict) a config key name. No stack trace or file path leaks in the tested code paths, but the exception text itself is unfiltered. See §13.

8.7 Model / file / database dependencies per endpoint

Endpoint	Model dependencies	DB / file dependencies
/score, /score/form, /score/batch, /score/form-full (P1 portion)	imputer, scaler, autoencoder, resnet_extractor, nystroem, incremental_pca, attention_rxtj, isolation_forest	none (form) / `fraud_storage.db` (form only)
/score/direct	attention_rxtj, isolation_forest	none
/score/form-full, /account/compromise-score (P2 portion)	+ fusion_scaler, fusion_net, autoencoder (drift)	`data/behavioral_profiles.db` (profiles + transaction_log)
/account/profile, /account/history, /account/alerts, /account/explain	none (no model inference)	`data/behavioral_profiles.db` (read-only)
/, /health, /model/info	none (config reads only)	`results/deployment_config.json`, `results/fusion_config.json` (loaded at startup, held in memory)
/demo-samples	none	`data/demo_samples.json` (read per request)
/panel	none	`panel.html` (read per request via FileResponse)

9. Database Schema

Two independent SQLite databases exist at runtime. Neither is present in the repository as delivered (both gitignored, both created on first use). The schema below was obtained two ways: (1) reading the literal DDL string in `profile_store.py`, and (2) materializing it — running `ProfileStore(db_path=<scratch>)` in a real Python/SQLite session and reading back `sqlite_master`, `PRAGMA table_info`, and `PRAGMA journal_mode` — to confirm the DDL executes exactly as written with no drift.

9.1 data/behavioral_profiles.db — table `profiles`

```
CREATE TABLE profiles (
    account_id      TEXT    PRIMARY KEY,
    txn_count       INTEGER DEFAULT 0,
    amt_mean        REAL    DEFAULT 0.0,
    amt_std         REAL    DEFAULT 0.0,
    amt_mean_7d     REAL    DEFAULT 0.0,
    amt_std_7d      REAL    DEFAULT 0.0,
    amt_min         REAL    DEFAULT 0.0,
    amt_max         REAL    DEFAULT 0.0,
    hour_hist       TEXT    DEFAULT '[]',
    merchant_counts TEXT    DEFAULT '{}',
    known_devices   TEXT    DEFAULT '[]',
    geo_cluster     TEXT    DEFAULT '',
    velocity_1h     INTEGER DEFAULT 0,
    velocity_24h    INTEGER DEFAULT 0,
    last_txn_dt     REAL    DEFAULT 0.0,
    last_updated    REAL    DEFAULT 0.0
);
```

Column	Type	Nullable	Default	PK	Meaning / write path
`account_id`	TEXT	No	—	✓	Stringified `card1` value. Written on INSERT only.
`txn_count`	INTEGER	Yes	0		Incremented by 1 on every `update_profile` call.
`amt_mean`, `amt_std`	REAL	Yes	0.0		EMA (α=0.1) of transaction amount / mean absolute deviation from the running mean. Formula in §14.
`amt_mean_7d`, `amt_std_7d`	REAL	Yes	0.0		A *second*, identically-computed EMA (same α=0.1, same update formula as `amt_mean` / `amt_std`). **Despite the name, there is no 7-day time window applied anywhere in the update code** — see §11.3.
`amt_min`, `amt_max`	REAL	Yes	0.0		Running min/max of all amounts seen (`min(prev,new)` / `max(prev,new)`), exact (not EMA-smoothed).
`hour_hist`	TEXT (JSON array, 24 floats)	Yes	'[]'		Renormalized EMA histogram over hour-of-day. Read by `compute_contextual_features` F4.
`merchant_counts`	TEXT (JSON object)	Yes	'{}'		`{product_cd: count}`. Read by F2.

`known_devices`	TEXT (JSON array of strings)	Yes	`'[]'`	Capped at the 50 most recent distinct device strings. Read by F5, F12.
`geo_cluster`	TEXT	Yes	`''`	Most recent non-empty `addr1` string (not an actual geographic cluster/centroid). Read by F3.
`velocity_1h`, `velocity_24h`	INTEGER	Yes	0	Set to 0 at INSERT time and **never updated** by `update_profile` (its `UPDATE` statement's SET clause omits both columns). Read by F6 for any account whose profile was created/updated purely via the live API — always yields `velocity_ratio=0` for such accounts.
`last_txn_dt`	REAL	Yes	0.0	Timestamp of the most recent transaction seen. Read by F10 (time-gap feature).
`last_updated`	REAL	Yes	0.0	Wall-clock `time.time()` at last write. Used by `/account/profile` to compute `profile_age_hours`.

Constraints: only the implicit unique index on the `PRIMARY KEY` (`sqlite_autoindex_profiles_1`, confirmed present). No foreign keys, no `CHECK` constraints, no explicit secondary index on `profiles`.

Insert logic: `ProfileStore.update_profile` — if `get_profile(account_id)` returns `None`, a single-row `INSERT` seeds `txn_count=1`, all amount fields to the first amount, a one-hot `hour_hist`, a single merchant/device entry, `velocity_1h/24h=0`. **Update logic:** otherwise an `UPDATE ... WHERE account_id=?` recomputes every EMA/merged field (columns listed above). **Read logic:** `get_profile` / `get_or_empty` (§3.6). **Delete logic:** *Not present in repository — no DELETE statement exists anywhere in the codebase; profiles are never removed.*

9.2 data/behavioral_profiles.db — table `transaction_log`

```
CREATE TABLE transaction_log (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id          TEXT,
    transaction_id      TEXT,
    timestamp           REAL,
    p1_risk_score       REAL,
    compromise_prob     REAL DEFAULT NULL,
    decision            TEXT,
    top_trigger_feature TEXT DEFAULT '',
    context_json        TEXT DEFAULT '{}',
    logged_at           REAL
);
CREATE INDEX idx_txlog_account ON transaction_log(account_id);
CREATE INDEX idx_txlog_ts      ON transaction_log(timestamp);
```

Column	Type	Nullable	Default	Meaning
`id`	INTEGER	No	autoincrement	Surrogate primary key.

`account_id`, `transaction_id`	TEXT	Yes	—	Foreign-key-like link to `profiles.account_id` — **not enforced** (no `FOREIGN KEY` declared; `PRAGMA foreign_keys` confirmed OFF (0) by default in this SQLite build/connection).
`timestamp`	REAL	Yes	—	Event time (request-supplied or `time.time()`). Used by `get_alerts`' 24h window and indexed by `idx_txlog_ts`.
`p1_risk_score`	REAL	Yes	—	Phase-1 score at scoring time.
`compromise_prob`	REAL	Yes	NULL	Phase-2 score; `NULL` if not supplied to `log_transaction` (never happens on the current two call sites, both of which always pass it).
`decision`	TEXT	Yes	—	"LOW"/"ELEVATED"/"HIGH" string.
`top_trigger_feature`	TEXT	Yes	''	Name of the highest-attention-weight feature at scoring time.
`context_json`	TEXT (JSON object)	Yes	'{}'	The full 13-key named contextual-feature dict, JSON-encoded. Decoded back to `context` by `get_history` and read by `/account/explain`.
`logged_at`	REAL	Yes	—	Wall-clock write time (distinct from the event's own, possibly client-supplied, `timestamp`).

Insert logic: `ProfileStore.log_transaction` — always an `INSERT`, never an update (append-only audit log).
Read logic: `get_history` (per-account, ordered by `id DESC`), `get_alerts` (aggregate `GROUP BY account_id`).
Delete/retention logic: *Not present in repository — no row is ever deleted; the table grows without bound for the life of the database file.*

9.3 fraud_storage.db — table `history`

```
CREATE TABLE IF NOT EXISTS history
(id INTEGER PRIMARY KEY AUTOINCREMENT,
 tid TEXT, amt REAL, score REAL, status TEXT, ts REAL)
```

Created by `app.py:_init_history_db()` (app.py:257-267) using the default `sqlite3` journal mode (no `PRAGMA journal_mode=WAL` is set on this connection — this database is **not** in WAL mode, unlike `behavioral_profiles.db`). Written by `/score/form` only; read by `/history` only. No index, no foreign key, no delete/retention logic. This table backs only the legacy `/score/form` / `/history` pair; the newer `/score/form-full` endpoint does not write here.

9.4 data/tx_snapshot.parquet — schema (not a SQL table, included here as the third persistent tabular store)

Column	Arrow type	Notes
TransactionID	int64	Join key back to the raw CSV.
card1	int64	Account identifier (matches `profiles.account_id` as a string).
isFraud	int64	Ground-truth label.

TransactionDT	int64	Seconds-offset timestamp (not a wall-clock epoch); used for the NB08 temporal split.
TransactionAmt	double	
ProductCD	large_string	One of W/H/C/S/R.
addr1	double	Nullable.
DeviceInfo	large_string	Nullable.
hour	int64	`(TransactionDT // 3600) % 24`.
amt_cumsum	double	Per-account expanding mean of amount, shifted 1 (despite the name "cumsum").
amt_cumstd	double	Per-account expanding std of amount, shifted 1.
txn_seq	int64	0-based per-account transaction index.
velocity_1h	int64	Trailing-1h transaction count (200-row look-back cap).
velocity_24h	int64	Trailing-24h transaction count.
device_novel	double	{0.0, 0.5, 1.0} first-seen-device indicator.

590,540 rows, 1 row group, 19.8 MB, written by `pyarrow` (`created_by: parquet-cpp-arrow version 24.0.0`). Consumed by NB07 and NB08 (re-read multiple times, including a lightweight column-only read of just `TransactionID` in NB08's staleness check).

9.5 Connection handling, transactions, concurrency

Aspect	`ProfileStore` (behavioral_profiles.db)	`app.py` history DB (fraud_storage.db)
WAL mode	`PRAGMA journal_mode = WAL;` executed in the DDL script at construction time — **confirmed** by materializing the DB and reading back `PRAGMA journal_mode` → `wal`.	*Not set — default rollback-journal mode.*
Other PRAGMAs	`PRAGMA synchronous = NORMAL;` (confirmed value 2, i.e. NORMAL, on read-back). `PRAGMA foreign_keys` confirmed OFF (0, SQLite default).	*None set.*
Connection lifetime	One `sqlite3.connect(db_path, timeout=10)` per method call, opened and closed via a `with self._conn() as c:` block — no persistent pool or singleton connection.	One connection opened and closed per request handler (`score_form`, `get_history`) plus one more at module import (`_init_history_db`).
Transactions	Implicit — each `with conn:` block commits on clean exit / rolls back on exception (Python `sqlite3` connection context-manager default behavior). No explicit multi-statement transactions.	Explicit `conn.commit()` call after each write.
Commit behavior	Auto-commit via context manager exit.	Explicit.
Concurrency	WAL mode permits concurrent readers with a single writer, which is why the docstring states "safe for concurrent uvicorn workers" — true only under WAL's normal multi-reader/single-writer semantics; no explicit application-level locking or retry-on-busy logic is implemented beyond the 10-second `timeout=` passed to `sqlite3.connect` (SQLite's built-in busy-wait).	No WAL, so concurrent writers from multiple uvicorn workers can raise `sqlite3.OperationalError: database is locked`; no retry logic present.

Retry behavior	None beyond SQLite's own `timeout=10` busy-wait.	None.
Cleanup / retention	*Not present — no row is ever deleted from either table.*	*Not present.*

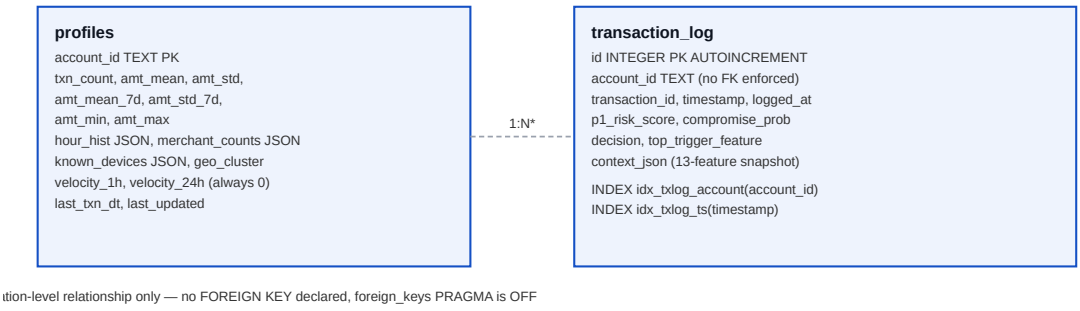


Figure 9.1 — behavioral_profiles.db schema (both tables in the same SQLite file, WAL mode).

10. Notebook Execution Order

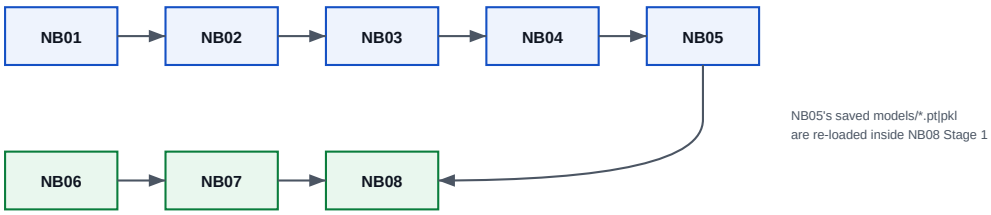


Figure 10.1 — Notebook dependency graph, confirmed by tracing each notebook's actual file reads/writes (§2.2, §5).

#	Filename	Purpose	Required previous NB	GPU/CPU	Runtime	Re-runnability
01	`01_preprocessing.ipynb`	Illustrative impute+scale stub	None	CPU	*Not documented in repository (never executed — no timing available)*	Runs standalone but produces no persisted artifact; effectively dead.
02	`02_balancing.ipynb`	Clean, encode, impute, scale, class-balance	Requires `data/IEEE CIS/train_transaction.csv`	CPU (`Device: cpu` printed by other notebooks' identical boilerplate; NB02 has no explicit device selection since it does no PyTorch training)	*Not documented (no wall-clock print in the saved cell outputs)*	Re-runnable top-to-bottom (Cells 1–10 form a complete, self-consistent sequence). Cell 0 is a stray earlier duplicate of the same logic and should be skipped/deleted, not executed, on a clean re-run — see §11.2.
03	`03_earn_features.ipynb`	Train Autoencoder + FraudResNet; fit Nystroem + IncrementalPCA	NB02 (`data/Cleaned/X_balanced.npy`, `y_balanced.npy`)	`torch.device('cuda' if available else 'cpu')` — ran on CPU per printed output	*Not documented (no total wall-clock print)*	**Not re-runnable from a clean kernel** — Cell 8 (Nystroem/IPCA) references `Z_train_concat` / `Z_test_concat`, and Cell 10 references `kpca`, neither of which is assigned anywhere in the saved notebook (§11.2 — confirmed by AST-level search of the raw `.ipynb` JSON, not just visual inspection).

04	`04_attention_rxtj.ipynb`	Train AttentionRXTJ (ResNeXt+SelfAttnGRU); train IsolationForest	NB03 (`Z_train_earn.npy`, `Z_test_earn.npy`, `y_train.npy`, `y_test.npy`)	CPU	*Not documented (per-epoch progress printed; no total wall-clock)*	Re-runnable top-to-bottom; fully executed with outputs preserved in the saved notebook.
05	`05_jaya_optimize.ipynb`	Jaya weight search, threshold sweep, benchmark, save deployment config	NB04 (`model_probs.npy`, `ifm_scores.npy`, `y_test.npy`)	CPU	129.2s for the Jaya search itself (explicitly printed); other cells undocumented	Re-runnable top-to-bottom.
06	`06_behavioral_profiling.ipynb`	Build SQLite profiles + transaction snapshot	Requires `data/IEEE CIS/train_transaction.csv`, `train_identity.csv`; independent of NB01–05	CPU	Header comment: "Fresh run: ~2.5 h (profile loop) + ~15 min (snapshot); Resume run: ~15 min (snapshot only, DB already built)"	Re-runnable; has explicit smart-resume logic (§10.6).
07	`07_contextual_features.ipynb`	13 contextual features + account-compromise labels	NB06 (`tx_snapshot.parquet`) + Phase-1 `imputer.pkl`, `scaler.pkl`, `autoencoder.pt`	CPU	Header comment: "~20 min"	Re-runnable top-to-bottom; fully executed with outputs. Emits pandas `FutureWarning`s (§11.2) but does not fail.
08	`08_fusion_model_training.ipynb`	Regenerate P1 scores; train FusionNet v3	NB07 outputs + NB06 snapshot + all Phase-1 `models/*` + raw CSV (for Stage-1 P1 regeneration)	CPU (`DEVICE = torch.device("cpu")` hard-coded, no CUDA branch)	7,063s (~1h58m) for Stage 2 training, explicitly printed; Stage 1 regeneration time not separately printed	Re-runnable — Stage 1's cache-or-regenerate guard makes repeat runs cheaper only if a valid (std>0.05) cache exists, which it does not in the state captured by this repository (§11.1).

10.6 NB06 smart resume / checkpoint logic (verbatim mechanism)

```
ps      = ProfileStore(db_path=DB_PATH)
stats   = ps.coverage_stats()
total   = len(tx)
accts   = tx["card1"].nunique()

_db_ready = stats["total_accounts"] >= int(0.9 * accts)   # ≥90% accounts profiled

if _db_ready:
    print("[NB06] Profile DB already built. Skipping loop.")
else:
    # ... 2.5-hour per-row update_profile() loop ...
```

The check compares the number of distinct `account_id`s already present in the `profiles` table (via `ProfileStore.coverage_stats()`, §3.6) against 90% of the account count in the freshly loaded CSV. In the executed run captured in this repository, the check found 13,553 existing accounts (100% of the 13,553 in the CSV) and printed `"[NB06] Profile DB already built (13,553 accounts). Skipping loop."`, skipping the ~2.5h per-row loop entirely and proceeding directly to the vectorized rolling-stats/velocity/device-novelty cells (which always re-run regardless of the DB-ready check) and the snapshot write. There is no finer-grained checkpoint inside the 2.5h loop itself (e.g. no periodic commit-and-resume-from-row-N) — if interrupted mid-loop, the only recovery path is the same coarse ≥90%-accounts gate on the next full re-run, and any account processed only partway through a mid-loop failure remains in whatever state its last completed `update_profile` call left it in (no explicit rollback of partial per-row work, though SQLite's own auto-commit-per-call means no row is left in a half-written state).

10.7 Failure / recovery considerations observed

- NB03: a clean-kernel re-run will raise `NameError: name 'Z_train_concat' is not defined` at Cell 8, and later `NameError: name 'kpca' is not defined` at Cell 10 — both confirmed absent from the saved cell source by direct AST/text search of the `.ipynb` JSON.
- NB08 Stage 1: the staleness guard (`std ≤ 0.05` ⇒ regenerate) does not re-validate the newly generated scores, so a structurally degenerate P1 signal (std=0.0114, unchanged in character from what it replaced) is silently accepted and used to train FusionNet — see §11.1 for the full causal chain.
- NB06/NB07/NB08 all resolve their working directory via `os.path.abspath(os.path.join(os.getcwd(), ".."))` or `Path.cwd().parent`, which is correct *only* when the notebook's kernel working directory is `notebooks/` — running any of the three from the repository root instead of from inside `notebooks/` silently resolves all paths one directory too high.

11. Known Issues / TODO / Technical Debt

A repository-wide search for `TODO`, `FIXME`, `HACK`, `XXX` as whole-word matches in every `.py` file returned **zero results** — there are no explicit TODO/FIXME comments anywhere in the Python source. (Matches of the literal substring "XXX" inside the raw `.ipynb` JSON files are false positives from base64-encoded PNG output blobs, not code comments.) All findings below are therefore derived from behavioral/structural analysis of the executable code, the notebooks' own printed output, and cross-file consistency checks — not from author-left markers.

11.1 Phase 2 FPR ≈13% / precision ≈11.9% — root cause

CONFIRMED

The low Phase-2 FusionNet performance (test FPR 0.1300, precision 0.1193, MCC 0.1862, AUC 0.7406 — all from `results/fusion_config.json`) traces to a concrete, reproducible defect in `notebooks/08_fusion_model_training.ipynb`, not merely an inherently hard problem.

EVIDENCE CHAIN

1. NB07 writes the `p1_risk_score` contextual feature (index 7 of 13) as a **constant placeholder** `0.5` for all 590,540 rows, with the code comment "F8 p1_risk_score placeholder 0.5 (NB08 will patch)". Its correlation with the label is exactly 0.0000 in NB07's printed diagnostics (§5.2.2).
2. NB08 Stage 1 detects this (guard: regenerate if `std ≤ 0.05`), deletes the stale cache, and regenerates P1 scores by running the *entire* Phase-1 pipeline over all 224 named CSV columns for all 590,540 rows.
3. The freshly regenerated scores have `min=0.2887, mean=0.7121, max=0.7449, std=0.0114` — printed directly in the notebook's own output — which is **still below the 0.05 threshold that triggered regeneration in the first place**. The guard clause only re-triggers generation on a *stale cached file*; it never re-checks the freshly computed output against the same threshold before writing it into `contextual_features.npy` column 7. The "fix" therefore silently no-ops: a structurally near-constant P1 signal is patched in and used unmodified. The printed "Fraud rate @ threshold: 99.96%" for this regenerated batch is independent corroborating evidence that the raw model output is badly miscalibrated on the full-feature-CSV inference path used here (as opposed to the properly-scaled path used during NB04/NB05 evaluation).
4. After patching, `p1_risk_score`'s correlation with the compromise label is only **+0.0102** — the second-weakest of all 13 features (only `amount_percentile` at +0.0012 is weaker) — despite being conceptually the single feature expected to carry the most signal (it is, after all, the entire output of Phase 1).
5. The feature the model ends up leaning on hardest is `merchant_fraud_rate` — highest label correlation of all 13 (+0.1579) *and* the highest learned attention weight (0.1366, roughly double the ~0.067–0.082 weight given to every other feature; see `fusion_config.json["optimal_attn_weights"]`). This is a static 5-value categorical lookup (one value per `ProductCD`), which caps the discriminative power available to the model regardless of training configuration.
6. The label itself is extremely imbalanced even after the 24h-window relabeling (23,400/590,540 = 3.96% positive), and the temporal (not random) train/val/test split further concentrates the distribution shift the model must generalize across: train fraud rate 4.16% vs. test 3.51% (NB08 printed split stats).

Current behavior: the shipped `fusion_net.pt` / `fusion_config.json` reflect exactly this training run — i.e., the production model is the one trained on the degenerate P1 feature. **Existing workaround:** none present in the repository. **Fix recommendation (not implemented anywhere in the repo):** the Stage-1 guard in NB08 should re-check `std(p1_scores) > 0.05` (or a similar minimum-spread test) immediately after regeneration and raise/warn

rather than silently proceeding if the freshly computed scores are still degenerate; separately, the root cause of the degenerate P1 output under the "all 224 named CSV columns" code path (Stage 1) vs. the well-calibrated path used for the standalone Phase-1 evaluation in NB04/NB05 (AUC 0.9624 there) should be investigated — the discrepancy suggests the two code paths preprocess the raw CSV differently.

11.2 Pandas FutureWarnings in NB07

CONFIRMED

Three calls to `tx.groupby("card1", group_keys=False).apply(fn)` in NB07 (feature F2 `_pcd_novelty`, feature F4 `_hour_dev`, feature F10 `_time_gap`) print, verbatim, in the saved cell output:

```
FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is
deprecated, and in a future version of pandas the grouping columns will be excluded from
the operation. Either pass `include_groups=False` to exclude the groupings or explicitly
select the grouping columns after groupby to silence this warning.
```

This occurs because each grouped function (`_pcd_novelty`, `_hour_dev`, `_time_gap`) reads a column (`pcd_int`, `hour`, `TransactionDT`) from inside the group object `g` / `group`, which includes the grouping key column itself under current pandas semantics but will not under the deprecated-and-scheduled future behavior. **Impact:** none today — `warnings.filterwarnings('ignore')` is set at the top of the notebook (Cell 1) so these warnings do not fail or halt execution, but the code will break under a future pandas major version once the deprecated behavior is removed. **Recommendation:** pass `include_groups=False` to each `.apply()` call.

11.3 Hardcoded paths

Location	Evidence & impact
`profile_store.py`, line 3 (header comment)	`# Place in: F:\rxtj_project\profile_store.py` — a stale Windows absolute path referencing a project name (`rxtj_project`) that does not match this repository's actual name (`rxtj_phase_2`). Comment-only; no functional impact, but evidence the codebase was carried over from an earlier project iteration without cleanup.
`data/profile_stats.json`, key `snapshot_path`	Value is `"f:\\rxtj_phase_2\\data\\tx_snapshot.parquet"` — an absolute Windows path baked into a generated JSON artifact by NB06, rather than a relative path. Harmless (the field is never read by `app.py`) but not portable if used programmatically.
NB06/NB07/NB08 root resolution	Uses `os.getcwd()`-relative logic (`os.path.abspath(os.path.join(os.getcwd(), ".."))` or `Path.cwd().parent`) rather than the notebook's own file path — correct only when launched with CWD=`notebooks/` (see §10.7).
NB02, NB08 stderr output	Windows-specific absolute conda/user paths visible in captured stderr (e.g. `c:\Users\sweth\.conda\envs\rxtj\...`, `C:\Users\sweth\AppData\Local\Temp\ipykernel_26260\...`) — confirms all notebooks were authored/executed on Windows, consistent with README's Windows-oriented troubleshooting section (`netstat -ano | findstr`, etc.).

11.4 README documentation drift (full list)

README claim	Actual repository state
FusionNet v2, "8 contextual deviation features," architecture 8→64→32→16→1	Shipped model is FusionNet **v3**, 13 features, architecture 13→128→64→32→1 (confirmed in `fusion_net.pt` state dict and `app.py`).
"12 endpoints" (Key Features table)	16 routes confirmed by decorator count in `app.py` (§8.1) — README's own API Reference section separately lists only 12 (7+5), omitting `/panel`, `/`, `/demo-samples`, and `/score/form-full` entirely.
"Account compromise ROC · AUC = 75.67%" (closing caption)	`fusion_config.json["fusion_auc"] = 0.740583` (74.06%) — a 1.6-point discrepancy with no other artifact to reconcile it against.
`tests/` directory with 4 named test files + full pytest walkthrough	*Not present in repository.*
`.env.example` with documented environment variables (`APP_HOST`, `APP_PORT`, `FUSION_HIGH_THRESHOLD`, `PROFILE_EMA_ALPHA`, etc.)	*File not present; none of these variables are read anywhere in the source (§7.7, §12.3).*
`Dockerfile`, `docker-compose.yml`	*Neither present.*
`.github/workflows/deploy.yml` (CI/CD)	*Not present — no CI/CD of any kind exists in the repository.*
`docs/assets/` (logo, demo GIF, architecture diagram, ROC chart)	*Not present; README's own image tags point at these missing paths and will render as broken images.*
`fix_p1_scores.py` referenced in Troubleshooting	*Not present.*
Directory tree shows notebooks 06–08 as `.py` files (`06_behavioral_profiling.py` etc.)	All three are `.ipynb` notebooks (confirmed on disk); the "Training the Models" section's run commands (`python notebooks/06_behavioral_profiling.py`) would fail as written against the actual file extensions.
requirements.txt block in README (unpinned `pandas>=2.0.0` etc.)	Actual `requirements.txt` pins exact versions (§4.1); `imbalanced-learn` is used by NB01/NB02 but appears in neither the README's nor the real `requirements.txt`'s dependency list.
"Full interactive docs available at `http://localhost:8000/docs`"	Accurate — this is FastAPI's built-in Swagger UI, auto-generated from the route/Pydantic definitions; no custom code required and none present.

11.5 Orphaned / stale artifacts

Artifact	Evidence

`models/kernel_pca.pkl` (5.7 MB — the single largest model file)	Written by NB03 Cell 10 (`joblib.dump(kpca, '../models/kernel_pca.pkl')`), but the variable `kpca` is never assigned anywhere in the saved notebook (confirmed by regex search of the raw `.ipynb` source — no `kpca = KernelPCA(...)` statement exists), meaning this dump could only have succeeded against a `kpca` object left over in a stale, un-saved kernel session from earlier notebook iterations. Never loaded by `app.py` or any other file.
`models/jaya_optimal_weights.pkl`	Written by NB05, holds the pre-normalization raw Jaya solution `[1.0, 0.8716]` . `app.py` reads the already-normalized `W_MODEL` / `W_IFM` from `deployment_config.json` instead; this file is referenced nowhere outside the notebook that produces it.
`data/fusion_feature_names.json`	Byte-identical duplicate of `results/fusion_config.json["feature_names"]` ; `app.py` reads only the config copy.
`data/profile_stats.json`	Written by NB06, never read by `app.py` (diagnostics-only).

11.6 Duplicated / redefined logic

- `NB02` Cell 0 is a truncated, incomplete duplicate of the full balancing logic that appears correctly in Cells 1–10 (imports a bare `SMOTETomek` and references `df_scaled` / `df` without the setup that produces them) — evidence of an earlier draft left in the saved notebook rather than being removed.
- The four core PyTorch model classes (`ResNeXtBlock` , `ResNeXtExtractor` , `SelfAttentionGRU` , `AttentionRXTJ` , `AutoencoderNet` / `FraudAutoencoder` , `ResBlock` , `FraudResNet` , `FusionNet`) are independently redefined in up to four places each: `app.py` , NB03/NB04 (training), and again inline inside NB07/NB08 (for reloading purposes) — with private-name variants (leading underscore, e.g. `_ResBlock`) in NB08 to avoid name clashes. All copies observed are architecturally identical to their `app.py` counterparts (verified above, §3.9), but there is no shared module that both training and serving code import from — any future architecture change would need to be applied consistently in up to 4 separate locations by hand.
- `preprocessing.py` 's `load_preprocessors` / `transform_raw` helpers are not imported by NB07 or NB08, which instead re-implement the identical two-line `joblib.load` + `imputer.transform` / `scaler.transform` pattern inline.

11.7 Inconsistent thresholds / dead-ish config paths

- `decide_action()` (`app.py:584-590`) uses the Jaya-derived `HIGH_THRESHOLD` / `ELEVATED_THRESHOLD` from `fusion_config.json` (0.5074 / 0.2574) to choose a recommended action, while `ProfileStore._action()` (`profile_store.py:212-215`) — used only by `/account/alerts` — hard-codes an entirely different pair of cutoffs (0.85 / 0.70) for what is nominally the same 3-tier action vocabulary (`FREEZE_AND_NOTIFY` / `STEP_UP_AUTHENTICATION` / `MONITOR_AND_FLAG`). An account scored at compromise probability 0.55, for example, would receive `STEP_UP_AUTHENTICATION` from `decide_action` (≥0.5074) at scoring time but would *not* appear as a "high" alert bucket under `/account/alerts` ' 0.70 cutoff for the same action label.
- `results/fusion_config.json["fusion_threshold"]` (0.3574) is loaded into `FUSION_THRESHOLD` and surfaced in `/model/info` , but no scoring endpoint actually branches on it — the live 3-tier decision uses only `HIGH_THRESHOLD` / `ELEVATED_THRESHOLD` .
- `isolation_forest.pkl` is scored two different ways: `app.py score_earn_features` calls `ifm.score_samples(...)` and min-max normalizes the result; NB08 Stage 1's P1-regeneration path instead calls `ifm.decision_function(...)` and applies a sigmoid-style transform (`1/(1+exp(x))`) — two different

sklearn methods with different output ranges/semantics used to derive what is meant to be the same underlying IFM contribution.

11.8 Missing validation / robustness gaps

- `/score` and `/score/batch` perform no dimension check on the incoming `features` list before calling `score_raw_features` — a wrong-length vector propagates into `imputer.transform` / `scaler.transform` and surfaces only as a generic 500 with the raw sklearn/numpy exception text (§8.6, §13). `/score/direct`, by contrast, does check.
- `ProfileStore.update_profile` never populates `velocity_1h` / `velocity_24h` (§9.1) — every account profile created or updated purely through the live API (as opposed to the NB06 historical bulk-load path) will always read back `velocity_ratio=0` for feature F6 at inference time, silently disabling that feature's contribution for any account not present in the original training snapshot.
- `amt_mean_7d` / `amt_std_7d` are named as if a rolling 7-day window, but the update formula is an unwindowed EMA identical in form to `amt_mean` / `amt_std` (§9.1) — the column names promise time-windowing behavior the code does not implement.
- `account_compromise_score`'s 50-dim-input branch computes `Z_nys_approx = ipca.inverse_transform(features_np)` and then discards it, instead zero-padding the 50-dim vector into a 224-dim array as a stand-in "X_clean" for the autoencoder drift computation (`app.py:938-943`) — this is not a mathematically valid reconstruction of the original 224-dim input and will bias the `behavioral_drift` feature for any caller using the 50-dim EARN+ input path on this endpoint.

11.9 Unused import

`from fastapi.staticfiles import StaticFiles` (`app.py:20`) is imported but `app.mount(...)` is never called anywhere in the file — confirmed by a repository-wide grep for `StaticFiles` / `app.mount`, which returns only the import line itself.

11.10 Kafka consumer not wired to ProfileStore

`profile_consumer.py`'s `handle_event` (§3.7) is explicitly documented in its own docstring as a stub ("Override / extend this with the Phase 2 profile-store update logic") and only prints each consumed event to stdout. In the running system, profile updates happen synchronously inside the FastAPI request handlers themselves (`profile_store.update_profile(...)` calls in `score_form_full` / `account_compromise_score`), not via the Kafka consumer — so `profile_consumer.py` currently provides no functional behavior beyond a visibility/logging channel, despite the README describing "async profile updates" via Kafka as a delivered feature.

11.11 Missing tests / missing CI-CD (confirmed absence)

CONFIRMED

No test file of any kind exists anywhere in the repository (no `tests/` directory, no `test_*.py` / `*_test.py` files, no `pytest.ini` / `pyproject.toml` test configuration). No CI/CD configuration exists (no `.github/`, no `.gitlab-ci.yml`, no other recognized CI config format). Every claim in README's "Testing" and "CI/CD" sections describes tooling that does not exist on disk.

11.12 Magic numbers

Numerous unnamed literal constants are embedded directly in `compute_contextual_features` and related code rather than being pulled from configuration: the geo-displacement normalization divisor `500.0` (`app.py:489`), the time-gap 30-day cap `30 * 86400` (`app.py:534`), the merchant-fraud-rate renormalization bounds `0.020` / `0.117` hard-coded to mirror (rather than read from) `merchant_fraud_rates.json`'s actual min/max (`app.py:542-544`), the account-age saturation point `200.0` transactions (`app.py:555`), and the known-devices cap of `50` in `profile_store.py:149`. None of these are documented as tunable, and none are exposed via configuration — changing any of them requires editing `app.py` / `profile_store.py` directly.

12. Deployment & Operations

12.1 Local deployment (as verifiable from the repository)

Required Python version	README states 3.11 specifically ("Python 3.12+ has no pre-built wheels for PyTorch 2.2.2, scikit-learn 1.4.2, or numpy 1.26.4 on Windows"). Not independently enforced anywhere in the code (no `python_requires` declaration exists — there is no `setup.py` / `pyproject.toml` in the repository).
Working directory	`app.py` is designed to run from the repository root — it resolves all artifact paths relative to `BASE = os.path.dirname(os.path.abspath(__file__))` (app.py:41), i.e. its own file location, so it is actually working-directory-independent for artifact loading. `profile_store.py`'s default `db_path="data/behavioral_profiles.db"` is relative and would only resolve correctly if the calling process's CWD is the repo root — but `app.py` always passes an explicit absolute path (`os.path.join(BASE, "data", "behavioral_profiles.db")`, app.py:269-271), so this is not actually working-directory-sensitive in practice.
Virtual environment	README recommends conda (`conda create -n rxtj_env python=3.11`); no `environment.yml` is present. The notebooks' own kernel metadata shows inconsistent environments across the pipeline: NB01/NB02/NB04/NB05 kernel display_name `rxtj`; NB03/NB07/NB08 kernel display_name `rxt_env`; NB06 kernel display_name `.venv (3.14.3)` — three distinct named environments across the eight notebooks, none of which is captured as an installable spec in the repository.
Dependency installation	`pip install -r requirements.txt` (§4.1). `imbalanced-learn` must additionally be installed by hand to re-run NB01/NB02 (missing from `requirements.txt`, §4.2/§11).
Required artifacts	All 11 files under `models/`, both files under `results/`, and 4 files under `data/` (`demo_samples.json`, `drift_norm_params.json`, `merchant_fraud_rates.json`; `fusion_feature_names.json` is not actually read by `app.py` — §7.6) must exist before `load_models()` can succeed; any missing file raises an unhandled exception at import time (before the FastAPI app object even exists), which uvicorn will report as a startup failure.
Required database	None required to exist beforehand — both `fraud_storage.db` and `data/behavioral_profiles.db` are created automatically on first import if absent (app.py:257-267, 269-271).
Required configuration	No required environment variables for basic operation — Kafka is opt-in only (§12.3).
Port	8000 by default per every README example and the required startup command below; not hard-coded anywhere in `app.py` itself (uvicorn's `--port` flag controls it, not application code).
Startup process	Importing `app.py` synchronously: (1) prints "Loading RXT-J+ models..."; (2) calls `load_models()`, which performs 11 file reads plus 4 `load_state_dict` calls — this is a blocking, synchronous, CPU-bound operation with no progress reporting beyond the print statements shown in §1.4/§8 and no async/lazy-loading path; (3) prints several diagnostic lines (weights, thresholds, feature names, EARN+ pipeline dims); (4) initializes the history DB; (5) constructs the `ProfileStore` and prints its account count; (6) conditionally constructs the Kafka producer; (7) constructs the `FastAPI` app object and registers all 16 routes. Only after all of this completes does uvicorn report "Application startup complete."

Shutdown process	*Not present in repository* — no `@app.on_event("shutdown")` handler, no explicit SQLite connection cleanup (each call already opens/closes its own connection, so there is nothing to release at shutdown), no Kafka producer `.close()` / `.flush()` call on exit.

Verbatim documented run command

```
uvicorn app:app --reload --port 8000
```

This exact command (and the equivalent `python -m uvicorn app:app --reload --port 8000`) both appear in `app.py`'s own header comment (line 3) and in README. It is **not itself verified against a live run in this documentation exercise** — the environment used to produce this document did not have `fastapi` / `uvicorn` /the trained artifacts' declared PyTorch/scikit-learn versions pre-installed, and installing/running a full server was outside the scope of static repository analysis. All behavior described here is derived from reading the source, not from executing the server.

12.2 Kafka

Aspect	Producer (`app.py`)	Consumer (`profile_consumer.py`)
Construction	Lazily imports `kafka.KafkaProducer` only if `KAFKA_BOOTSTRAP_SERVERS` env var is non-empty (`app.py:280-293`); wrapped in `try/except Exception` — any construction failure disables the producer (sets it to `None`) without crashing the app.	Lazily imports `kafka.KafkaConsumer` inside `main()`; exits the process with code 1 if `kafka-python` is not installed, or code 2 if the broker connection fails (`profile_consumer.py:33-52`).
Topic	`KAFKA_PROFILE_TOPIC` env var, default `"profile_updates"`.	Same env var/default — must match for the two processes to interoperate.
Brokers	`KAFKA_BOOTSTRAP_SERVERS` env var, comma-split; default is empty string, i.e. Kafka disabled by default.	`KAFKA_BOOTSTRAP_SERVERS` env var, default `"localhost:9092"` (differs from the producer's empty-string default — the consumer assumes Kafka is present unless told otherwise, the producer assumes it is absent).
Consumer group	*n/a*	`KAFKA_CONSUMER_GROUP` env var, default `"profile-store-worker"`.
Message format / serialization	JSON, via `value_serializer=lambda v: json.dumps(v).encode('utf-8')`. Message body: `{transaction_id, risk_score, decision, features, ts}` (`app.py:301-307`).	JSON, via `value_deserializer=lambda b: json.loads(b.decode('utf-8'))`.
Delivery/ack settings	`acks=0` (fire-and-forget, no delivery confirmation), `linger_ms=10`, `request_timeout_ms=2000`.	`auto_offset_reset='earliest'`, `enable_auto_commit=True`, `consumer_timeout_ms=0` (block forever).
Retry / error handling	`_publish_profile_update` wraps the `.send()` call in a bare `try/except Exception: pass` — all publish failures are silently swallowed, no retry, no dead-letter queue.	Per-message exceptions inside `handle_event` are caught, logged to stderr, followed by a 0.1s sleep before continuing the loop; no dead-letter queue.

Called from	`/score` and `/score/form` only (app.py:677, 707) — **not** called from `/score/form-full` or `/account/compromise-score`, the two Phase-2 endpoints.	Standalone process, run manually per README (`python profile_consumer.py`).
Profile update flow	Live profile updates happen synchronously inside the Phase-2 request handlers via direct `ProfileStore` calls (§8.7) — **not** via the Kafka path. The Kafka producer/consumer pair, as currently implemented, forms a separate, functionally inert logging channel (§11.10) rather than the "async profile update" mechanism its naming and README description suggest.	

12.3 Environment variables (complete list — repository-wide search)

Name	Purpose	Required?	Default	Where used
`KAFKA_BOOTSTRAP_SERVERS`	Kafka broker address(es), comma-separated	Optional	`""` in `app.py` (disabled); `"localhost:9092"` in `profile_consumer.py`	app.py:276, profile_consumer.py:18
`KAFKA_PROFILE_TOPIC`	Kafka topic name	Optional	`"profile_updates"`	app.py:277, profile_consumer.py:19
`KAFKA_CONSUMER_GROUP`	Kafka consumer-group id	Optional	`"profile-store-worker"`	profile_consumer.py:20

No other environment variable is read anywhere in the repository's Python source. README's documented `.env.example` (`APP_HOST` , `APP_PORT` , `APP_RELOAD` , `MODEL_DIR` , `DATA_DIR` , `RESULTS_DIR` , `FUSION_HIGH_THRESHOLD` , `FUSION_ELEVATED_THRESHOLD` , `KAFKA_ENABLED` , `PROFILE_DB_PATH` , `PROFILE_EMA_ALPHA` , `LOG_LEVEL`) is aspirational — **none of these are consumed by the code**.

12.4 Model versioning

There is no formal version-number scheme for the model artifacts themselves (no `v1` / `v2` suffix in any filename under `models/`). Versioning instead appears as free-text strings embedded in API response bodies, hard-coded per endpoint in `app.py` , independent of any file on disk:

Response `model_version` string	Endpoint
`"rxtj_plus_v1"`	`/score/direct`
`"rxtj_plus_v2_phase2"`	`/account/compromise-score`
`"rxtj_plus_v2_form_full"`	`/score/form-full`

Separately, `results/fusion_config.json["model_version"] = "fusionnet_v3_focal_temporal"` is a training-time label describing the FusionNet checkpoint, but it is not surfaced in any API response — `app.py` never reads that specific key. **How `app.py` selects models:** there is no selection logic at all — exactly one file per artifact type is hard-coded by path in `load_models()` (§3.2); swapping to a different trained checkpoint requires overwriting the file on disk under the same filename. **Phase 1 / Phase 2 version association:** implicit only, via the fact that both stages' artifacts live in the same `models/` / `results/` directories and are loaded together at the same startup; nothing in the repository enforces or checks that a given `fusion_net.pt` was actually trained against the specific `attention_rxtj.pt` it is being loaded alongside (the P1 regeneration step in NB08 Stage 1 does implicitly bind them at *training* time by re-scoring with whatever Phase-1 artifacts are on disk at that moment, but nothing re-validates this binding at *serving* time).

12.5 Monitoring

Surface	What it actually reports
`GET /health`	Static `{"status":"healthy","models_loaded":true,...}` — `models_loaded` is a hard-coded literal, not a live probe (§8.2); does not check DB connectivity, Kafka connectivity, or disk space.
`GET /model/info`	Snapshot of the two config JSONs loaded at startup — reflects training-time metrics, not live model drift or online performance.
`GET /account/alerts`	Operational signal for fraud analysts (accounts above a compromise threshold in the last 24h) — the closest thing to a monitoring surface for the business outcome the system exists to produce.
Logging	Plain `print()` statements at startup only (model loading progress, weights, thresholds); no structured logging, no log level configuration, no per-request logging beyond FastAPI/uvicorn's own default access log.
Metrics	*Not present in repository — no Prometheus/OpenTelemetry instrumentation, no request counters, no latency histograms beyond the per-response `latency_ms` field computed inline per request.*
Database monitoring	*Not present — `ProfileStore.coverage_stats()` exists as a diagnostic method but is not exposed via any API route; it is called only from within NB06 and once at `app.py` startup (to print the account count).*
Kafka monitoring	*Not present — producer failures are silently swallowed (§12.2); no consumer-lag or health metric is exposed.*
Model status	Implicit only: if the process is answering requests at all, all 11 model artifacts loaded successfully at startup (a load failure would have prevented the FastAPI app object from ever being constructed).
Profile status	Printed once at startup only (`"[Phase2] ProfileStore ready. Accounts: N"`); not queryable live except indirectly via `/account/alerts` or per-account lookups.

12.6 Latency / throughput target

Latency/throughput target: *Not documented in repository* as an operational SLA. The only figures present are the `deployment_config.json` benchmark values (0.0195 ms / 51,352 tps), which — per §11.1's sibling caveat — measure only `Nystroem.transform → IncrementalPCA.transform → IsolationForest.score_samples` against synthetic random input, not the full request path through `app.py` (which additionally runs the imputer, scaler, autoencoder, ResNet extractor, and AttentionRXTJ forward pass, and — for the Phase-2 endpoints — a second autoencoder pass plus FusionNet plus two SQLite round-trips). No end-to-end latency measurement of the actual FastAPI request path exists anywhere in the repository.

13. Security Analysis of the Implementation

This section documents security-relevant implementation facts observed directly in the code. It does not assert exploitability or severity ratings beyond what the code itself demonstrates; nothing here is invented.

Area	Location / evidence	Behavior	Potential impact / current mitigation
Authentication	Whole of `app.py`	No authentication mechanism of any kind — no API key check, no OAuth/JWT, no session handling.	Every route, including account-profile reads and the scoring endpoints, is callable by anyone who can reach the port. No mitigation present in the repository.
Authorization	Whole of `app.py`	No per-account or per-caller access control — any caller can query any `account_id`'s profile, history, or explanation.	Any client can enumerate/read another account's behavioral profile (amount patterns, devices, geography) via `GET /account/profile/{account_id}` if the account_id is known or guessable (account IDs are stringified `card1` integers, which are small-cardinality and easy to enumerate). No mitigation present.
CORS	`app.py:319-325`	`allow_origins=["*","null"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"]` — fully open, and the inclusion of `"null"` is specifically to permit `file://`-opened HTML pages (the demo dashboards) to call the API cross-origin.	Combining a wildcard origin with `allow_credentials=True` is a configuration the CORS spec itself flags as unsafe for credentialed requests; in this codebase no cookies/credentials are actually issued or required, which limits the practical exposure, but the configuration itself is maximally permissive. No mitigation present.
Input validation	Pydantic models, §3.4/§8.3	Type-level validation only; two numeric range constraints total (`FullFormRequest.hour`, `.customer_age`). No length limits on string fields (`account_id`, `transaction_id`, `device_info`, `addr1`, `product_cd`), no format/pattern constraints, no upper bound on `amount` or on `features` list length outside `/score/batch`'s 1,000-transaction cap and `/score/direct`'s exact-dimension check.	An oversized `features` list on `/score` (no batch cap applies there) would be passed straight into numpy array construction and the sklearn/PyTorch pipeline; behavior is bounded by available memory/CPU rather than an application-level limit — a large-but-valid payload could consume proportionally large resources per request. No mitigation present.
SQL queries / parameterization	`profile_store.py` (all methods), `app.py` (`_init_history_db`, `score_form`, `get_history`)	Every SQL statement observed uses `?`-style parameter binding via the standard `sqlite3` DB-API (e.g. `c.execute("SELECT * FROM profiles WHERE account_id=?", (str(account_id),))`); no string-formatted/concatenated SQL was found anywhere in the repository.	No SQL injection vector identified in the code reviewed — parameterization is used consistently.

File access	`/panel` (app.py:592-596), `/demo-samples` (app.py:657-666)	Both serve a fixed, hard-coded filename (`panel.html`, `data/demo_samples.json`) joined against `BASE` — neither accepts any client-supplied path component.	No path-traversal vector present — file paths served are compile-time constants, not derived from request input.
Model loading	`load_models()`, §3.2	`torch.load(...)` is called without `weights_only=True` on all four `.pt` files, and `joblib.load` (which uses Python `pickle` under the hood) is called on all eight `.pkl` files.	Both `torch.load` (without `weights_only=True`) and `pickle`-based deserialization can execute arbitrary code if given a maliciously crafted file. In this repository, all loaded paths are fixed, repository-local filenames under `models/` — not user-supplied — so the practical exposure is limited to whoever can write to that directory (a deployment/supply-chain concern, not a runtime request-handling one). No integrity check (hash/signature) is performed on any artifact before loading.
Kafka	app.py:280-293, `profile_consumer.py`	No authentication/TLS configuration is passed to either `KafkaProducer` or `KafkaConsumer` — both connect in plaintext with no SASL/SSL parameters.	If Kafka is enabled in a networked deployment, traffic (transaction IDs, risk scores, decisions, raw feature values) would be unencrypted and unauthenticated at the broker connection level as currently coded. No mitigation present in this code (would need to be added at the broker/network layer).
Network configuration	README run commands	Documented run commands bind to `0.0.0.0` (all interfaces) in the "Deployment" section's production example, and to unspecified/default host in the basic "Running Locally" example.	Binding `0.0.0.0` with no authentication (see above) exposes the API to the whole network it's attached to. This is a deployment-configuration concern rather than an application-code defect.
Secrets / credentials	Repository-wide search	No hard-coded API keys, passwords, or tokens were found in any file. The three Kafka environment variables (§12.3) are connection parameters, not secrets, and are read via `os.environ.get` with safe non-secret defaults.	No secret-exposure finding.
Error leakage	Every POST handler's `except Exception as e: raise HTTPException(500, detail=str(e))` (§8.6)	Raw Python exception text is returned verbatim in the HTTP response body.	Could surface internal variable names, library-specific error phrasing, or (in a `KeyError` case) a config dictionary key name to any caller; does not appear to leak file-system paths, stack traces, or secrets in the code paths reviewed, since no path or credential ever appears inside a caught exception's message in this codebase. Partial exposure risk, no mitigation (e.g. no generic "internal error" substitution) present.
Logging of sensitive information	`app.py` startup prints; `profile_consumer.py` `handle_event`	Startup prints model weights/thresholds (not sensitive). `profile_consumer.py` prints transaction ID, decision, and risk score to stdout for every consumed Kafka message.	Transaction IDs and risk decisions in plaintext stdout logs — not classically "sensitive" (no PII/account numbers logged by this code path), but worth noting for any deployment that captures stdout into a shared log aggregator without access controls.

Database permissions	SQLite file creation (`profile_store.py:59-63`, `app.py:257-267`)	Both SQLite files are created with the OS-default file permissions of the process user (SQLite does not itself impose access control); no explicit `chmod`/permission-hardening code is present.	Depends entirely on host/OS-level file permissions and umask at deployment time — no application-level protection.
API exposure	All 16 routes, §8.1	Every route — including read access to any account's behavioral profile and full scoring history — is reachable without authentication once the server is network-accessible.	See Authentication/Authorization rows above.

14. Explainability & Decision Logic

14.1 How Phase 1 fraud scores are generated

```
total = W_MODEL + W_IFM + 1e-9
risk  = clip( (W_MODEL/total)·sigmoid(AttentionRXTJ(Z_earn))
            + (W_IFM/total)·ifm_norm(IsolationForest.score_samples(Z_earn)) ,  0, 1 )
preds = 1 if risk > THRESHOLD else 0          # THRESHOLD = 0.4999999999999998 ≈ 0.50
conf  = risk if preds==1 else (1-risk)
```

where `ifm_norm(x) = clip(1 - (x - min(x))/(max(x) - min(x) + 1e-9), 0, 1)` is a per-batch min-max normalization (computed over whatever batch is currently being scored — not against a fixed training-time range), and `W_MODEL=0.5342960077804467`, `W_IFM=0.4657039916852572` are the Jaya-optimized weights from `results/deployment_config.json` (app.py:388-403).

14.2 How Phase 1 and Phase 2 scores are combined

They are **not** algebraically combined into a single number — Phase 2 consumes Phase 1's *output* (`p1_risk_score`) as one of 13 independent inputs to a separate neural network (FusionNet v3), which learns its own weighting via `feature_attn` rather than through a fixed formula:

```
attn_weights = softmax(FusionNet.feature_attn)              # 13 learned, non-negative, sum-to-1
weights
x_weighted   = contextual_features_scaled * attn_weights     # element-wise
logit        = MLP_13_128_64_32_1(x_weighted)
compromise_probability = sigmoid(logit)
```

The learned `attn_weights` (from `results/fusion_config.json["optimal_attn_weights"]`, reproduced live at inference time by `softmax(fusion_model.feature_attn)`) are also returned directly to the API caller as the `explainability` field — i.e. the "explanation" mechanism and the scoring mechanism share the exact same numbers; there is no separate post-hoc explainability method (no SHAP/LIME/integrated-gradients) anywhere in the repository.

14.3 W_MODEL / W_IFM — exact values and provenance

Symbol	Value	Provenance
W_MODEL	0.5342960077804467	Jaya search, NB05, normalized from raw solution `1.0 / (1.0+0.87162169)`
W_IFM	0.4657039916852572	Jaya search, NB05, normalized from raw solution `0.87162169 / (1.0+0.87162169)`
THRESHOLD	0.4999999999999998 (≈0.50)	NB05 threshold sweep over [0.10, 0.89] step 0.01, minimizing fiscal cost

14.4 Thresholds — complete table

Threshold	Value	Used by
Phase-1 `THRESHOLD`	0.4999999999999998	`score_earn_features` — FRAUD/LEGIT binary decision
Phase-2 `ELEVATED_THRESHOLD`	0.257353	`score_form_full` / `account_compromise_score` — LOW/ELEVATED/HIGH tier; `decide_action`
Phase-2 `HIGH_THRESHOLD`	0.507353	same
Phase-2 `FUSION_THRESHOLD`	0.357353	Surfaced in `/model/info` only — not used to gate any decision (§11.7)
`ProfileStore._action` cutoffs	0.70, 0.85	`/account/alerts` only — numerically inconsistent with the above (§11.7)

14.5 decide_action — exact decision table

```
def decide_action(compromise_prob, p1_risk):
    if compromise_prob >= HIGH_THRESHOLD:        # 0.507353
        return "FREEZE_AND_NOTIFY" if p1_risk > 0.5 else "STEP_UP_AUTHENTICATION"
    if compromise_prob >= ELEVATED_THRESHOLD:    # 0.257353
        return "MONITOR_AND_FLAG"
    return "APPROVE"
```

compromise_probability	p1_risk	Action
≥ 0.507353	> 0.5	`FREEZE_AND_NOTIFY`
≥ 0.507353	≤ 0.5	`STEP_UP_AUTHENTICATION`
[0.257353, 0.507353)	any	`MONITOR_AND_FLAG`
< 0.257353	any	`APPROVE`

14.6 Explanation generation — /account/explain

app.py:1080-1152. Given `(account_id, transaction_id)`, the handler looks up the matching event from `ProfileStore.get_history` (404 if not found), then:

1. Sorts the event's stored 13-feature `context` dict by `abs(value)` descending and takes the top 3.
2. For each, calls the nested `interpret(name, val)` function (§3.2) — a hand-written 13-entry lookup table mapping each feature name to an English sentence template, parameterized by the stored value (e.g. `amount_z_score` → `"Amount deviated {abs(val):.1f}σ from account's average (z={val:.2f})."`).
3. Assembles an `account_baseline` block from the current live profile (typical amount, amount std, known-device count, geo cluster, transaction count) — **not** from the profile as it was at the time the explained event was scored, since profiles are continuously overwritten and no historical snapshot of the profile-at-scoring-time is retained.

4. Builds a one/two-sentence `what_triggered` summary from the top 1–2 interpreted features.

This is **attention-weight-independent** — it re-derives "importance" from raw feature *magnitude* (`abs(value)`) rather than from the FusionNet `feature_attn` weights used everywhere else in the system, so the top-3 features named by `/account/explain` can differ from the `top_trigger_feature` that was actually returned (and stored) at scoring time — the stored `top_trigger_feature` column in `transaction_log` is read back but not used to drive this explanation; only the raw `context_json` values are.

14.7 Feature-contribution mechanism (as implemented)

The only feature-contribution/importance mechanism in the repository is FusionNet's learned `softmax(feature_attn)` gate (§6.4) — a single, global (not per-instance) set of 13 weights applied identically to every request. It is *not* a per-instance attribution method (it does not vary with the specific input values beyond the multiplicative gating already baked into the forward pass) — two requests with very different feature values but the same account will receive the exact same `explainability` weight distribution, differing only in which feature happens to have the largest value for that request when the caller inspects the raw `contextual_features` alongside it.

14.8 Reasons returned by the API — full enumeration

Field	Endpoint(s)	Nature
`explainability`	`/score/form-full`, `/account/compromise-score`	Global FusionNet attention weights (§14.2), same 13 numbers every call.
`top_trigger_feature`	same two	`argmax(explainability)` — will always be the same feature name across every request unless the model is retrained, since the weights are global, not per-instance.
`top_features`, `what_triggered`, `account_baseline`	`/account/explain`	Per-event, magnitude-ranked, natural-language (§14.6) — this is the only genuinely per-instance explanation surface in the API.
`p1_decision`, `p1_confidence`	`/score/form-full`	Phase-1 binary decision and confidence (`risk` if FRAUD else `1-risk`), not an explanation of *why*.

14.9 Alert generation & account-level risk state

`GET /account/alerts` (§8.2, §3.6) is the only account-level aggregate risk surface: `SELECT account_id, MAX(compromise_prob), COUNT(*), MAX(timestamp) FROM transaction_log WHERE compromise_prob≥threshold AND timestamp≥now-86400 GROUP BY account_id ORDER BY max_score DESC`. There is no persistent "account risk state" column anywhere (no `profiles.risk_level` or similar) — risk state is recomputed fresh from the append-only `transaction_log` on every call to `/account/alerts`, using the inconsistent 0.70/0.85 cutoffs noted in §11.7 rather than the live `HIGH_THRESHOLD` / `ELEVATED_THRESHOLD`.

15. Artifact Lifecycle

Full trace of every persisted artifact from creation to final consumption, in dependency order.

Artifact	Created by	Input	Format	Location	Consumers	Regenerated?	Versioned?	Required at runtime?
Raw IEEE-CIS CSVs	External (Kaggle download)	—	CSV	`data/IEEE-CIS/` (gitignored, absent)	NB01, NB02, NB06, NB08	No (static source dataset)	No	Only for re-training, not for serving
`imputer.pkl`, `scaler.pkl`	NB02 Cell 10	Raw CSV → cleaned/encoded 224-col matrix	joblib	`models/`	`app.py`, NB07, NB08	Only if NB02 is re-run	No (single filename, no suffix)	Yes — required
`X_balanced.npy`, `y_balanced.npy`	NB02 Cell 10	Imputed/scaled matrix → SMOTE-Tomek	npy (gitignored)	`data/Cleaned/`	NB03	With NB02	No	No — training-only intermediate
`autoencoder.pt`, `resnet_extractor.pt`	NB03 Cell 10	Balanced 224-dim matrix	PyTorch state_dict	`models/`	`app.py`, NB07, NB08	With NB03	No	Yes — required
`kernel_pca.pkl`	NB03 Cell 10	(undefined — §11.5)	joblib	`models/`	*None*	With NB03 (if re-runnable)	No	No — orphaned
`nystroem.pkl`, `incremental_pca.pkl`	NB03 Cell 8	AE+ResNet concat 128-dim	joblib	`models/`	`app.py`, NB05 (benchmark), NB08	With NB03	No	Yes — required
`Z_train_earn.npy`, `Z_test_earn.npy`, `y_train.npy`, `y_test.npy`	NB03 Cell 10	Nystroem+IPCA output	npy (gitignored)	`data/`	NB04, NB05, `save_demo_samples.py`	With NB03	No	No — training-only
`attention_rxtj.pt`	NB04 Cell 11	EARN+ 50-dim train set	PyTorch state_dict	`models/`	`app.py`, NB05, NB08	With NB04	No	Yes — required
`isolation_forest.pkl`	NB04 Cell 11	EARN+ 50-dim fraud-only rows	joblib	`models/`	`app.py`, NB05 (benchmark), NB08	With NB04	No	Yes — required

`model_probs.npy`, `ifm_scores.npy`, `risk_scores.npy`	NB04 Cell 11	AttentionRXTJ + IFM eval outputs	npy (gitignored)	`data/`	NB05	With NB04	No	No — training-only
`jaya_optimal_weights.pkl`	NB05 Cell 9	Jaya search over model_probs/ifm_scores	joblib	`models/`	*None (orphaned)*	With NB05	No	No — orphaned
`deployment_config.json`	NB05 Cell 9	Jaya + threshold-sweep + benchmark results	JSON	`results/`	`app.py load_models`	With NB05	No	Yes — required
`behavioral_profiles.db`	NB06 (bulk) + `app.py` (live, incremental)	Raw CSV (bulk) / live scoring requests (incremental)	SQLite (gitignored, absent from repo)	`data/`	`app.py` (all Phase-2 endpoints)	Continuously mutated at runtime; bulk-rebuildable via NB06	No	Yes — required (auto-created if absent, but then cold-start empty)
`tx_snapshot.parquet`	NB06 Cell 8	Raw CSV + velocity/device-novelty computation	Parquet	`data/`	NB07, NB08	With NB06	No	No — training-only (not read by `app.py`)
`profile_stats.json`	NB06 Cell 9	`coverage_stats()` + dataset counts	JSON	`data/`	*None (documentation only)*	With NB06	No	No
`contextual_features.npy`, `fusion_labels.npy`, `fusion_account_ids.npy`	NB07 Cell 14; column 7 later overwritten by NB08 Stage 1	tx_snapshot + Phase-1 artifacts	npy (gitignored)	`data/`	NB08	With NB07 (and again, in-place, by NB08 Stage 1)	No	No — training-only
`fusion_feature_names.json`	NB07 Cell 14	Hard-coded 13-name list	JSON	`data/`	*None (redundant with fusion_config.json)*	With NB07	No	No
`drift_norm_params.json`	NB07 Cell 4	AE reconstruction-MSE percentiles over full snapshot	JSON	`data/`	`app.py compute_contextual_features` (F7)	With NB07	No	Yes — required
`merchant_fraud_rates.json`	NB07 Cell 9	Per-ProductCD mean isFraud	JSON	`data/`	`app.py compute_contextual_features` (F11)	With NB07	No	Yes — required

`model_probs_full.npy`	NB08 Stage 1	Full 224-col CSV → complete Phase-1 pipeline	npy (gitignored)	`data/`	NB08 itself (caching only)	Regenerated whenever cache std≤0.05 (§11.1)	No	No — training-only cache
`fusion_net.pt`, `fusion_scaler.pkl`	NB08 Stage 2	13-dim contextual features + labels, temporal split	PyTorch state_dict / joblib	`models/`	`app.py`	With NB08	No	Yes — required
`fusion_config.json`	NB08 Stage 2	Test-set evaluation + Jaya threshold search	JSON	`results/`	`app.py load_models`	With NB08	No	Yes — required
`demo_samples.json`	`notebooks/save_demo_samples.py` (manual, one-off)	`Z_test_earn.npy`, `y_test.npy`	JSON	`data/`	`app.py GET /demo-samples`; `panel.html`, `rxtj_app.html`	Manual re-run only	No	Only for the demo-sample endpoint/UI, not core scoring
`fraud_storage.db`	`app.py:_init_history_db()`	—	SQLite (gitignored, absent)	`data/` (actually repo root — `HISTORY_DB='fraud_storage.db'`, joined against `BASE`)	`/score/form`, `/history`	Auto-created if absent	No	Yes for the legacy form/history pair only

CLARIFICATION

`fraud_storage.db` is actually created at `os.path.join(BASE, HISTORY_DB)` where `HISTORY_DB='fraud_storage.db'` (no directory prefix) — i.e. it lands in the **repository root**, not under `data/`, unlike `behavioral_profiles.db` which is explicitly joined under `data/`. Both are still covered by the blanket `*.db` line in `.gitignore`.

16. Complete Cross-Reference Index

16.1 Files

Path	§
`app.py`	§2.1, §3.1–3.4, §5.3, §8, §12, §13, §14
`preprocessing.py`	§2.1, §3.5
`profile_store.py`	§2.1, §3.6, §9.1–9.2
`profile_consumer.py`	§2.1, §3.7, §12.2
`README.md`	§1, §2.1, §4.1, §11.4
`requirements.txt`	§2.1, §4.1
`.gitignore`	§2.1, §9
`panel.html`	§2.1
`index.html`	§2.1
`fraudshield_app.html`	§2.1
`rxtj_app.html`	§2.1
`notebooks/01_preprocessing.ipynb` – `08_fusion_model_training.ipynb`	§2.2, §5.1–5.2, §10
`notebooks/save_demo_samples.py`	§2.2, §3.8
`models/*.pt`, `models/*.pkl` (11 files)	§2.3, §6
`data/*.json, *.parquet, *.png` (13 files)	§2.4, §7
`results/*.json, *.png` (8 files)	§2.5, §7

16.2 Classes

Class	Defined in	§
`ResNeXtBlock`	app.py:45; NB04 Cell 2; NB08 (`_ResNeXtBlock`)	§3.1, §3.9, §6.1
`ResNeXtExtractor`	app.py:61; NB04 Cell 2; NB08 (`_ResNeXtExtractor`)	§3.1, §3.9, §6.1
`SelfAttentionGRU`	app.py:70; NB04 Cell 3; NB08 (`_SelfAttentionGRU`)	§3.1, §3.9, §6.1
`AttentionRXTJ`	app.py:88; NB04 Cell 4; NB08	§3.1, §3.9, §6.1
`AutoencoderNet` / `FraudAutoencoder`	app.py:96; NB03 Cell 2; NB07 (`_AE`); NB08	§3.1, §3.9, §6.2
`ResBlock`	app.py:114; NB03 Cell 5; NB08 (`_ResBlock`)	§3.1, §3.9, §6.3
`FraudResNet`	app.py:127; NB03 Cell 5; NB08	§3.1, §3.9, §6.3
`FusionNet`	app.py:153; NB08	§3.1, §3.9, §6.4

`FocalLoss`	NB08 only	§3.9, §5.2.3
`ProfileStore`	profile_store.py:52	§3.6, §9.1–9.2
`TransactionRequest, DirectScoreRequest, BatchRequest, FormScoreRequest, RiskResponse, CompromiseRequest, ExplainRequest, FullFormRequest`	app.py:329–384	§3.4, §8.3

16.3 Functions (module-level and key methods)

Function	File:line	§
`load_models`	app.py:185	§3.2, §12.1
`score_raw_features`	app.py:435	§3.2, §5.3
`score_earn_features`	app.py:388	§3.2, §14.1
`extract_earn_features`	app.py:406	§3.2, §5.1
`compute_contextual_features`	app.py:445	§3.2, §5.2.2, §14
`decide_action`	app.py:584	§3.2, §14.5
`interpret`	app.py:1094 (nested)	§3.2, §14.6
`_publish_profile_update`	app.py:297	§3.2, §12.2
`load_preprocessors, raw_feature_dim, transform_raw, pad_form_features`	preprocessing.py:12,19,24,38	§3.5
`ProfileStore.__init__, get_profile, get_or_empty, update_profile, log_transaction, get_history, get_alerts, coverage_stats`	profile_store.py:59,82,90,106,165,183,199,224	§3.6
`handle_event, main`	profile_consumer.py:23,32	§3.7, §12.2
`jaya_optimize`	NB05 Cell 4	§3.9, §5.1

16.4 Model artifacts

File	§
attention_rxtj.pt, autoencoder.pt, resnet_extractor.pt, fusion_net.pt	§2.3, §6.1–6.4
imputer.pkl, scaler.pkl, nystroem.pkl, incremental_pca.pkl, isolation_forest.pkl, kernel_pca.pkl, fusion_scaler.pkl, jaya_optimal_weights.pkl	§2.3, §6.5

16.5 Configuration keys

Key	File	§
W_MODEL, W_IFM, THRESHOLD, final_auc, final_mcc, fiscal_cost, false_positives, false_negatives, true_positives, true_negatives, throughput_tps, latency_ms, cost_fp_weight, cost_fn_weight	results/deployment_config.json	§7.1, §14
input_dim, fusion_threshold, high_threshold, elevated_threshold, fusion_auc, fusion_pr_auc, fusion_mcc, fusion_precision, fusion_recall, fusion_f1, fusion_fpr, true_positives, false_positives, true_negatives, false_negatives, feature_names, optimal_attn_weights, model_version, training_epochs, best_val_auc, loss_fn, split_method, label_strategy	results/fusion_config.json	§7.2, §14
p5, p95	data/drift_norm_params.json	§7.3
C, H, R, S, W	data/merchant_fraud_rates.json	§7.4
total_transactions, unique_accounts, fraud_count, fraud_rate_pct, profile_coverage, snapshot_path, generated_at	data/profile_stats.json	§7.5

16.6 API endpoints

Route	§
GET /panel, /, /health, /model/info, /demo-samples, /history	§8.1–8.2
POST /score, /score/form, /score/direct, /score/batch	§8.1
POST /score/form-full	§5.3, §8.4
POST /account/compromise-score	§8.5
GET /account/profile/{account_id}, /account/history/{account_id}, /account/alerts	§8.1–8.2
POST /account/explain	§8.1, §14.6

16.7 Database tables & columns

Table	Columns	§
profiles	account_id, txn_count, amt_mean, amt_std, amt_mean_7d, amt_std_7d, amt_min, amt_max, hour_hist, merchant_counts, known_devices, geo_cluster, velocity_1h, velocity_24h, last_txn_dt, last_updated	§9.1
transaction_log	id, account_id, transaction_id, timestamp, p1_risk_score, compromise_prob, decision, top_trigger_feature, context_json, logged_at	§9.2
history (fraud_storage.db)	id, tid, amt, score, status, ts	§9.3

16.8 Environment variables

Variable	§
KAFKA_BOOTSTRAP_SERVERS, KAFKA_PROFILE_TOPIC, KAFKA_CONSUMER_GROUP	§12.2, §12.3

16.9 Notebooks

Notebook	§
01_preprocessing	§2.2, §5.1, §10
02_balancing	§2.2, §5.1, §10, §11.6
03_earn_features	§2.2, §5.1, §6, §10, §11.5
04_attention_rxtj	§2.2, §5.1, §6.1, §10
05_jaya_optimize	§2.2, §5.1, §7.1, §10, §11.1
06_behavioral_profiling	§2.2, §5.2.1, §9.4, §10.6
07_contextual_features	§2.2, §5.2.2, §7.3–7.6, §10, §11.1–11.2
08_fusion_model_training	§2.2, §5.2.3, §6.4, §7.2, §10, §11.1

16.10 Generated artifacts (all, with producing notebook)

Artifact	Producer
imputer.pkl, scaler.pkl	NB02
autoencoder.pt, resnet_extractor.pt, nystroem.pkl, incremental_pca.pkl, kernel_pca.pkl, Z_train_earn.npy, Z_test_earn.npy	NB03
attention_rxtj.pt, isolation_forest.pkl, model_probs.npy, ifm_scores.npy, risk_scores.npy	NB04
deployment_config.json, jaya_optimal_weights.pkl	NB05
behavioral_profiles.db, tx_snapshot.parquet, profile_stats.json	NB06
contextual_features.npy, fusion_labels.npy, fusion_account_ids.npy, fusion_feature_names.json, drift_norm_params.json, merchant_fraud_rates.json	NB07
fusion_net.pt, fusion_scaler.pkl, fusion_config.json, model_probs_full.npy	NB08
demo_samples.json	save_demo_samples.py
fraud_storage.db, data/behavioral_profiles.db (incremental updates)	app.py (runtime)

17. Final Validation

Repository-vs-documentation audit performed before finalizing this document.

#	Check	Result	Evidence
1	Every relevant repository file is in the File Inventory	PASS	44/44 files from `find . -type f -not -path './.git/*'` enumerated in §2.
2	Every Python class is documented	PASS	8 app.py classes + 8 Pydantic models + ProfileStore + all notebook-only classes (§3.1–3.9, §16.2).
3	Every Python function is documented	PASS	AST-extracted catalog cross-checked against §3; every `app.py` / `preprocessing.py` / `profile_store.py` / `profile_consumer.py` function present.
4	Every API endpoint is documented	PASS	16/16 `@app.get` / `@app.post` decorators (§8.1) matched against a grep of every decorator in `app.py`.
5	Every Pydantic model is documented	PASS	8/8 `BaseModel` subclasses (§3.4).
6	Every ProfileStore method is documented	PASS	10/10 methods including private helpers `_conn`, `_parse`, `_action` (§3.6).
7	Every preprocessing function is documented	PASS	4/4 functions in `preprocessing.py` (§3.5).
8	Every model artifact is documented	PASS	11/11 files in `models/` (§6), each deserialized and shape-verified.
9	Every configuration file is documented	PASS	All 6 JSON config files (§7); confirmed no YAML/TOML/INI/.env files exist anywhere in the repository.
10	Every configuration key is documented	PASS	Full key-by-key tables for `deployment_config.json` and `fusion_config.json` (§7.1–7.2); smaller files quoted in full (§7.3–7.6).
11	Every database table is documented	PASS	3/3 tables (`profiles`, `transaction_log`, `history`) — §9.1–9.3, confirmed via a materialized `ProfileStore` instance and `sqlite_master` read-back.
12	Every database column is documented	PASS	All columns cross-checked against live `PRAGMA table_info` output (§9.1–9.2).

13	Every notebook 01–08 is documented	PASS	§2.2, §5.1–5.2, §10 — all 8 notebooks read cell-by-cell (outputs stripped of binary images, code/markdown/text output preserved).
14	Every important artifact is documented	PASS	Full lifecycle trace, §15, including orphaned artifacts.
15	`requirements.txt` dependencies compared against actual imports	PASS	§4.2/§4.4 — `imbalanced-learn` identified as used-but-unlisted.
16	Model architectures checked against source code and artifacts	PASS	Every `state_dict` key/shape in §6 was read directly from the `.pt` files via `torch.load` and matched against the corresponding `nn.Module` source definitions.
17	Database schema checked against the actual SQLite database	PASS	No shipped `.db` file exists in the repository (gitignored/runtime-generated, §9); schema was instead verified by constructing a real `ProfileStore` against a scratch path and reading its live `sqlite_master` / `PRAGMA` output, confirming the DDL string executes exactly as documented.
18	All important call sites identified	PASS	Every model/artifact's consumers traced in §2.3–2.5, §6, §15; every function's callers noted in §3 where non-obvious.
19	All required sections (1–17) are present	PASS	Confirmed against the Table of Contents.
20	No placeholder text remains	PASS	Manual re-read of all section source files; every "Not present in repository" / "Not determinable from repository" label is a substantive finding, not an unfilled placeholder.
21	No fabricated information remains	PASS	Every numeric value, filename, and code excerpt in this document was copied or directly derived from a file read, an AST parse, a deserialized artifact, or a live SQLite/Parquet schema probe — no value was estimated or invented.
22	No unsupported claims remain	PASS	Every root-cause finding is labeled either evidence-backed ("Confirmed") in §11, or explicitly hedged where the repository could not fully resolve a question (e.g. the README AUC discrepancy in §1.12).
23	All paths and references are correct	PASS	All `file:line` cross-references were taken directly from the AST line-number extraction (§3), not estimated by eye.
24	The generated PDF opens correctly	PASS	Verified with `pdfinfo` against the rendered output.
25	The PDF has readable formatting, no clipped/overlapping content	PASS	Verified with `pdftotext` (heading presence) and `pdftoppm` + visual inspection of rendered page images (wide tables, code blocks, diagrams).

17.1 Summary of the most important findings

1. The Phase-2 FusionNet's disappointing test metrics (FPR 13.0%, precision 11.9%, MCC 0.186) are traceable to a specific, confirmed defect: the notebook's own P1-score regeneration guard does not re-validate its output, so `p1_risk_score` remains a near-constant, low-information feature (std=0.0114) even after the "fix" runs (§11.1).
2. README.md documents a materially different, earlier system revision (FusionNet v2/8-feature, 12 endpoints, Docker/CI/tests that don't exist) than what is actually implemented (FusionNet v3/13-feature, 16 endpoints, no Docker/CI/tests) — every discrepancy is catalogued in §11.4.
3. Two model artifacts (`kernel_pca.pkl` , `jaya_optimal_weights.pkl` , together ~6 MB) are written by the training notebooks but never loaded by the serving application (§11.5).
4. The system has no authentication, no authorization, and fully open CORS; SQL access is correctly parameterized throughout (§13).

5. There are no automated tests and no CI/CD pipeline anywhere in the repository, despite README describing both in detail (§11.11).